BLUEPRINT PROJEKTU · WERSJA 0.2

Silnik likwidacji

Stablecoin dolarowy jako stanowisko badawcze: koncepcja, specyfikacja, plan i wdrożenie testowe

Protokół CDP z kotwicą dolarową, zbudowany po to, by wyprowadzić parametry ryzyka z danych i porównać dwa mechanizmy likwidacji w identycznych warunkach — na forku Base z prawdziwą wyrocznią i prawdziwą płynnością z momentów krachu.

TYP	Blueprint: koncepcja, specyfikacja techniczna, plan, wdrożenie testowe
NAZWA	Redoubt (nazwa robocza)
ŚRODOWISKA	Anvil lokalnie, fork Base mainnet, Base Sepolia; bez realnych środków
STAN WIEDZY	Narzędzia i protokoły referencyjne sprawdzone w październiku 2026
STATUS	Wersja 0.2 po rewizji założeń; parametry liczbowe są hipotezami

Wersja 0.2 zastępuje 0.1. Dodano specyfikację techniczną kontraktów, wyczerpujący plan krok po kroku, szczegółowy stos technologiczny oraz procedurę wdrożenia testowego. Plan jest zależnościowy, bez estymat czasowych. Liczby w dokumencie pochodzą z prostych modeli i są punktem wyjścia, nie wynikiem projektu.

Spis treści

1. Streszczenie . . . 6
1.1 Najważniejsze decyzje w pigułce . . . 6
1.2 Co powstanie . . . 7
1.3 Jak czytać ten dokument . . . 7
2. Dlaczego likwidacje zawodzą . . . 8
2.1 Cztery źródła długu nieściągalnego . . . 8
2.2 Specyfika warstwy drugiej . . . 8
3. Cele i zakres . . . 9
3.1 Cel nadrzędny . . . 9
3.2 Cele techniczne . . . 9
3.3 Poza zakresem . . . 9
3.4 Kryteria sukcesu . . . 9
4. Kluczowe koncepcje . . . 11
4.1 Pozycja i wskaźniki . . . 11
4.2 Minimalny dług i rezerwa gazowa . . . 11
4.3 Odsetki i ich przepływ . . . 11
4.4 Kotwica: podłoga i sufit . . . 11
4.5 Likwidacja pełna i częściowa . . . 11
4.6 Wodospad pokrywania strat . . . 12
4.7 Tryb awaryjny i podział pro rata . . . 12
4.8 Role uczestników . . . 12
5. Historia rewizji . . . 13
5.1 Rewizja przy wersji 0.1 . . . 13
5.2 Rewizja przy wersji 0.2 . . . 13
6. Decyzje projektowe . . . 14
7. Architektura . . . 15
7.1 Widok ogólny . . . 15
7.2 Kontrakty i odpowiedzialności . . . 16
7.3 Granice odpowiedzialności . . . 16
8. Specyfikacja techniczna kontraktów . . . 17
8.1 Konwencje liczbowe i zaokrąglenia . . . 17

8.2 Kluczowy stan . . . 17
8.3 Odsetki . . . 18
8.4 Lista pozycji i kolejka wykupu . . . 18
8.5 Pula stabilności: algorytm produkt–suma . . . 18
8.6 Redystrybucja . . . 19
8.7 Aukcja holenderska . . . 19
8.8 OracleAdapter . . . 20
8.9 Tryb awaryjny . . . 20
8.10 Zdarzenia . . . 20
8.11 Macierz uprawnień . . . 21

9. Cykl życia pozycji . . . 22
9.1 Scenariusze . . . 22

10. Wyrocznia i budżet opóźnień . . . 24
10.1 Składniki opóźnienia . . . 24
10.2 Pierwsze przybliżenie bufora . . . 24
10.3 Bufor z apetytu na ryzyko . . . 25
10.4 Składniki minimalnego wskaźnika zabezpieczenia . . . 25
10.5 Obrona przed złą ceną . . . 25

11. Mechanizmy likwidacji . . . 26
11.1 Pula stabilności (wdrożenie A) . . . 26
11.2 Aukcja holenderska (wdrożenie B) . . . 26
11.3 Redystrybucja i deficyt . . . 26
11.4 Opłacalność likwidacji . . . 26
11.5 Osie porównania . . . 27

12. Model protokołu i symulator . . . 28
12.1 Dwie implementacje tych samych reguł . . . 28
12.2 Moduły . . . 28
12.3 Pętla symulacji . . . 29
12.4 Czego model nie udowadnia . . . 29

13. Dane i kalibracja . . . 30
13.1 Zestawy danych . . . 30
13.2 Pomiar płynności na forku . . . 30
13.3 Kalibracja likwidatorów i błąd przeżywalności . . . 30
13.4 Scenariusze . . . 31

14. Metoda badawcza i metryki . . . 32
14.1 Prerejestracja . . . 32
14.2 Metryki . . . 32
14.3 Raportowanie . . . 32
14.4 Szkielet raportu końcowego . . . 32
15. Środowiska i wdrożenie testowe . . . 33
15.1 Anvil lokalnie . . . 33
15.2 Fork Base mainnet: tryb historyczny . . . 34
15.3 Fork Base mainnet: tryb odtwarzany . . . 34
15.4 Base Sepolia: procedura krok po kroku . . . 34
15.5 Klucze i sekrety . . . 35
15.6 Oznaczenia i granice wdrożenia testowego . . . 35
16. Strategia testów . . . 36
16.1 Warstwy . . . 36
16.2 Niezmienniki . . . 36
16.3 Testy stanowe w Foundry . . . 37
16.4 Dowody symboliczne . . . 37
16.5 Test różnicowy . . . 37
16.6 Ciągła integracja . . . 38
17. Bezpieczeństwo i model zagrożeń . . . 39
17.1 Założenia zaufania . . . 39
17.2 Rejestr zagrożeń . . . 39
17.3 Granice . . . 39
18. Stos technologiczny i infrastruktura . . . 40
18.1 Kontrakty . . . 40
18.2 Model, dane i symulacja . . . 40
18.3 Operacje i publikacja . . . 40
18.4 Repozytorium . . . 41
19. Plan realizacji krok po kroku . . . 42
19.1 Etap 0: fundamenty . . . 42
19.2 Etap 1: dane . . . 43
19.3 Etap 2: rdzeń kontraktów . . . 43
19.4 Etap 3: pula i redystrybucja (wdrożenie A) . . . 44
19.5 Etap 4: model i test różnicowy . . . 44
19.6 Etap 5: kalibracja i pierwsze wyniki (MVP) . . . 45

19.7 Etap 6: aukcja (wdrożenie B) . . . 45
19.8 Etap 7: tryb awaryjny . . . 46
19.9 Etap 8: hartowanie i wdrożenia . . . 46
19.10 Etap 9: publikacja . . . 46

20. Praca z Claude Code . . . 48
20.1 Zasady . . . 48

21. Mocne i słabe strony . . . 49
21.1 Mocne strony . . . 49
21.2 Słabe strony . . . 49
21.3 Ryzyka . . . 49

22. Demonstracja i publikacja . . . 50
22.1 Przebieg demonstracyjny (1 minuta) . . . 50
22.2 Wersja rozszerzona i publikacja . . . 50

23. Otwarte pytania . . . 51

24. Słownik pojęć . . . 52

25. Źródła i materiały . . . 53
25.1 Protokoły referencyjne . . . 53
25.2 Infrastruktura i narzędzia . . . 53
25.3 Bezpieczeństwo, ryzyko i dane . . . 53

A. Dodatek: wzory i parametry . . . 54

B. Dodatek: szkic interfejsów . . . 55

C. Dodatek: konfiguracja scenariusza . . . 56

D. Dodatek: szkic skryptu wdrożenia . . . 57

E. Dodatek: szablon prerejestracji . . . 57

1 Streszczenie

Protokoły pożyczkowe oparte na zabezpieczeniu kryptowalutowym stoją na jednym założeniu: gdy wartość zabezpieczenia spadnie poniżej progu, ktoś szybko zamknie pozycję. Gdy to założenie zawodzi, protokół zostaje z długiem nieściągalnym, a stablecoin traci pokrycie. Parametry, które mają temu zapobiec — minimalny wskaźnik zabezpieczenia, premia likwidacyjna, rozmiar puli, krzywa aukcji — są w większości projektów przepisywane z innych protokołów.

> **Teza projektu**
>
> Budujemy minimalny protokół CDP z kotwicą dolarową po to, żeby **wyprowadzić parametry ryzyka z danych dla jawnie określonego apetytu na ryzyko** i porównać dwa mechanizmy likwidacji w identycznych warunkach. Kontrakty są nośnikiem tezy; produktem końcowym jest odtwarzalny raport.

Protokół jest celowo mały: jedno zabezpieczenie (WETH), jeden stablecoin, stała stopa, wykup po nominale, brak governance i ról administracyjnych. Strategia likwidacji jest niezmiennym parametrem wdrożenia, więc porównanie odbywa się na dwóch wdrożeniach tego samego rdzenia: z pulą stabilności i z aukcją holenderską. Redystrybucja i księga deficytu domykają wodospad strat, a tryb awaryjny kończy się podziałem zabezpieczenia pro rata — bez wyścigu i bez zależności od ceny.

Najważniejsza zmiana względem wersji 0.1 dotyczy środowiska eksperymentów. Testnet nie ma prawdziwej płynności ani prawdziwych likwidatorów, więc nie może weryfikować ekonomii. Główne eksperymenty odbywają się na **forku Base mainnet na historycznych blokach** — z prawdziwą wyrocznią i prawdziwymi pulami DEX z momentów krachu. Base Sepolia pozostaje publiczną wizytówką: zweryfikowane kontrakty, działający bot, monitor niezmienników.

1.1 Najważniejsze decyzje w pigułce

Obszar	Decyzja	Dlaczego
Kotwica	Dolar	Kotwica ma nie dokładać problemów; trudność ma siedzieć w likwidacjach
Zabezpieczenie	Tylko WETH	Najwięcej danych historycznych, brak ryzyka korelacji
Strategia likwidacji	Parametr niezmienny wdrożenia; porównanie na dwóch wdrożeniach	Brak funkcji administracyjnych i mniej powierzchni ataku
Małe pozycje	Minimalny dług i rezerwa gazowa przy otwarciu	Pozycja nieopłacalna do likwidacji nie może powstać
Odsetki	Stała stopa; część odsetek zasila pulę stabilności	Pula musi mieć z czego żyć
Wykup	Po nominale, kolejka według NICR	Twarda podłoga kotwicy; kolejność niezależna od ceny
Tryb awaryjny	Podział zabezpieczenia pro rata, w naturze	Brak arbitrażu starej ceny i brak runu
Wyrocznia	Chainlink + feed dostępności sekwencera	Na L2 to warunek uczciwych likwidacji
Eksperymenty	Fork Base mainnet na historycznych blokach	Prawdziwa wyrocznia i płynność z momentu krachu
Metoda	Prerejestracja, blokowy bootstrap, kalibracja na protokołach z Base	Wyniki do obrony, a nie do dopasowania
Publiczne wdrożenie	Base Sepolia, token z sufiksem TEST	Weryfikowalność bez udawania produktu

1.2 Co powstanie

- **Kontrakty** (Solidity, Foundry): rdzeń CDP, lista pozycji według NICR, wykup, dwie strategie likwidacji, księga deficytu, tryb awaryjny.
- **Model protokołu w Pythonie**, zgodny z kontraktami dzięki testowi różnicowemu w CI.
- **Potok danych**: ceny, historia wyroczni na Base, zdarzenia likwidacji, pomiary płynności z forków — z kartami źródeł.
- **Symulator** z modelami wyroczni, płynności i likwidatorów oraz generatorem ścieżek.
- **Raport** z mapami parametrów, wyprowadzonym buforem i porównaniem mechanizmów.
- **Wdrożenia**: skrypty dla Anvila, forka i Base Sepolia, bot likwidacyjny, monitor niezmienników.
- **Materiały**: README, wideo, wpis techniczny.

1.3 Jak czytać ten dokument

Rozdziały	Zawartość
2–6	Problem, cele, koncepcje, historia rewizji, decyzje projektowe
7–11	Architektura, specyfikacja techniczna kontraktów, cykl życia, wyrocznia, mechanizmy likwidacji
12–14	Model i symulator, dane i kalibracja, metoda badawcza i metryki
15–16	Środowiska i wdrożenie testowe, strategia testów
17–18	Bezpieczeństwo, stos technologiczny i infrastruktura
19–20	Plan realizacji krok po kroku, praca z Claude Code
21–25	Ocena projektu, demo, otwarte pytania, słownik, źródła
A–E	Wzory, interfejsy, konfiguracje, skrypty wdrożeniowe, szablony

2 Dlaczego likwidacje zawodzą

Dług nieściągalny nie powstaje dlatego, że „rynek spadł". Powstaje wtedy, gdy spadek ceny wyprzedzi zdolność systemu do zamknięcia pozycji. Źródła tego wyprzedzenia są cztery i każde jest w tym projekcie osobnym parametrem.

2.1 Cztery źródła długu nieściągalnego

Źródło	Mechanika	Przykład
Opóźnienie informacji	Protokół widzi cenę z wyroczni, aktualizowaną przy progu odchylenia albo po heartbeacie. W gwałtownym ruchu jest systematycznie spóźniona.	Każdy gwałtowny krach
Brak opłacalności	Likwidacja ma koszt. Jeśli premia nie pokrywa gazu, poślizgu i kosztu kapitału, racjonalny bot jej nie wykona.	Marzec 2020: przeciążona sieć i opłaty sparaliżowały likwidatorów, część aukcji w MakerDAO wygrano ofertami bliskimi zeru
Brak płynności	Likwidator musi sprzedać zabezpieczenie; przy dużej pozycji poślizg zjada premię, a sprzedaż pogłębia spadek.	Kaskady likwidacji na rynkach lewarowanych
Awaria infrastruktury	Zepsuta wyrocznia, zamrożony rynek, niedostępny sekwencer L2.	Maj 2022: manipulacja ceną LUNA i dług nieściągalny w protokołach pożyczkowych

Trzy z czterech źródeł nie mają nic wspólnego z poprawnością kodu. Kontrakt może być bezbłędny, a protokół i tak straci pieniądze, bo parametry nie przewidziały realnego zachowania rynku i botów. To jest przestrzeń, którą bada ten projekt.

2.2 Specyfika warstwy drugiej

Na Base koszt transakcji jest o rzędy wielkości niższy niż na Ethereum, więc bariera opłacalności przesuwa się z gazu na poślizg. Pojawia się też ryzyko, którego na L1 nie ma: pojedynczy sekwencer. Gdy jest niedostępny, użytkownik nie dopłaci zabezpieczenia, a po wznowieniu protokół zobaczy cenę po ruchu. Dlatego feed dostępności sekwencera i okres karencji są częścią rdzenia, a symulacja ma dwa reżimy kosztów.

> **Uwaga o danych historycznych**
>
> Krachy z marca 2020 i maja 2022 wydarzyły się, zanim Base istniał. Ich **ścieżki cenowe** są przenośne i służą jako wejście, ale **zachowanie infrastruktury** — gaz, wyrocznie, likwidatorzy — z tamtych dni to zjawiska Ethereum L1. Dla Base korzystamy ze scenariuszy natywnych, takich jak gwałtowny spadek z 5 sierpnia 2024 roku, dla którego istnieje prawdziwy stan łańcucha.

3 Cele i zakres

3.1 Cel nadrzędny

Pokazać umiejętność rzadką i dobrze płatną w DeFi: **łączenie inżynierii kontraktów z ilościową analizą ryzyka**. Projekt ma dowodzić, że autor potrafi napisać bezpieczny kontrakt i odpowiedzieć liczbami na pytanie, przy jakich warunkach ten kontrakt przestanie chronić protokół.

3.2 Cele techniczne

ID	Cel	Co to znaczy w praktyce
G1	Poprawny rdzeń CDP	Pozycje, odsetki, wykup, likwidacja i tryb awaryjny z księgowością potwierdzoną niezmiennikami
G2	Dwa wdrożenia do porównania	Ten sam rdzeń z pulą stabilności i z aukcją holenderską
G3	Zgodność modelu z kontraktami	Test różnicowy w CI na losowych scenariuszach
G4	Realne otoczenie	Eksperymenty na forku Base z prawdziwą wyrocznią i płynnością
G5	Skalibrowany model likwidatorów	Rozkład opóźnień odtworzony z danych protokołów na Base, z opisanym błędem przeżywalności
G6	Wyprowadzone parametry	Bufor i próg opłacalności jako funkcje apetytu na ryzyko, zmienności i płynności
G7	Rzetelna metoda	Prerejestracja hipotez przed pierwszym przemiataniem
G8	Publiczne wdrożenie testowe	Zweryfikowane kontrakty na Base Sepolia, bot i monitor

3.3 Poza zakresem

- **Mainnet i realne środki.** Bez audytu zewnętrznego obsługa cudzych pieniędzy byłaby nieodpowiedzialna.
- **Dynamika kotwicy.** Wykup jest w kontraktach jako mechanizm poprawności, ale analiza peg wymaga modelu rynku stablecoina i arbitrażystów — to rozszerzenie.
- **Wiele zabezpieczeń, stopy ustawiane przez użytkowników, governance, token zarządczy.**
- **Własna wyrocznia** — korzystamy z istniejącej i modelujemy jej ograniczenia.
- **Emisja jako produkt** — to demonstracja techniczna, także ze względu na regulacje dotyczące emitentów stablecoinów.

3.4 Kryteria sukcesu

Kryterium	Jak mierzymy	Próg docelowy
Księgowość	Kampania fuzzingu niezmienników	0 naruszeń przy co najmniej 10 mln wywołań
Pokrycie rdzenia	forge coverage	co najmniej 95% linii i gałęzi
Zgodność modelu	Test różnicowy	zgodność do tolerancji zaokrągleń na ≥ 200 scenariuszach
Kalibracja	Porównanie rozkładów opóźnień	rozkład z symulacji w przedziale ufności danych
Rzetelność	Prerejestracja w historii repozytorium	commit przed pierwszym przemiataniem
Wyprowadzenie parametrów	Raport	każdy parametr rozbity na składniki z uzasadnieniem
Wdrożenie publiczne	Base Sepolia	kontrakty zweryfikowane, test dymny przechodzi

Kryterium	Jak mierzymy	Próg docelowy
Odtwarzalność	Jedno polecenie	raport i wykresy z czystego stanu

Progi są celami pomiaru. Wynik publikujemy niezależnie od tego, czy zostaną osiągnięte.

4 Kluczowe koncepcje

4.1 Pozycja i wskaźniki

Pozycja to zdeponowane zabezpieczenie i wyemitowany dług. **ICR** to wartość zabezpieczenia podzielona przez dług, **TCR** — ten sam wskaźnik dla całego systemu, **MCR** — minimalny dopuszczalny ICR. Dodatkowo używamy **NICR**, czyli nominalnego wskaźnika: zabezpieczenie podzielone przez dług znormalizowany, bez ceny. Ponieważ wszystkie pozycje mają to samo zabezpieczenie i tę samą stopę, kolejność pozycji według NICR jest identyczna z kolejnością według ICR, ale nie zmienia się przy ruchu ceny ani przy naliczaniu odsetek. To pozwala utrzymywać posortowaną listę pozycji bez przeliczania jej po każdej aktualizacji wyroczni.

4.2 Minimalny dług i rezerwa gazowa

Każda pozycja musi mieć dług nie mniejszy niż ustalone minimum, a przy otwarciu wpłaca niewielką rezerwę w WETH, która trafia do likwidatora jako rekompensata kosztu transakcji albo wraca do właściciela przy zamknięciu. Dzięki temu nie istnieją pozycje, których likwidacja byłaby nieopłacalna — problem rozwiązany konstrukcyjnie, a nie premią rosnącą dla małych pozycji, jak zakładała wersja 0.1.

4.3 Odsetki i ich przepływ

Dług rośnie według globalnego indeksu, a dług pozycji to jej dług znormalizowany pomnożony przez bieżący indeks. Naliczone odsetki są emitowane jako stablecoin: ustalona część trafia do puli stabilności jako wynagrodzenie depozytariuszy, reszta do rezerwy protokołu pokrywającej deficyt. Ten przepływ jest jawny, bo bez niego „koszt kapitału puli” nie miałby źródła, a pula w symulacji byłaby fikcją.

4.4 Kotwica: podłoga i sufit

Strona	Mechanizm	Arbitraż
Podłoga	Wykup po nominale: stablecoin zawsze wymienialny na zabezpieczenie warte dolara, minus opłata	Poniżej dolara opłaca się kupić i wykupić
Sufit	Swobodna emisja po koszcie odsetek	Powyżej dolara opłaca się emitować i sprzedawać

Wykup jest w rdzeniu jako mechanizm poprawności: gwarantuje podłogę i podlega niezmiennikom. Analiza dynamiki kotwicy wymagałaby jednak modelu rynku stablecoina — puli handlowej i arbitrażystów — i dlatego jest poza zakresem MVP.

4.5 Likwidacja pełna i częściowa

Model pełny zamyka pozycję w całości; częściowy zamyka ustalony ułamek długu. U nas ułamek domknięcia jest parametrem wdrożenia i dodatkową osią porównania.

4.6 Wodospad pokrywania strat



Rysunek 1. Kolejność pokrywania straty z pozycji bez pokrycia.

Każda strata musi dać się przypisać do szczebla wodospadu. Osobną kategorią księgową jest **dług w aukcji**: w chwili startu aukcji dług opuszcza pozycję i przechodzi do księgi deficytu jako zobowiązanie oczekujące na rozliczenie. Jest wliczany do TCR i niezmienników aż do zakupu albo uznania za stratę. **Dług nieściągalny** to część długu, której po zamknięciu pozycji nie pokryło zabezpieczenie, pula ani redystrybucja.

4.7 Tryb awaryjny i podział pro rata

Tryb awaryjny włącza się automatycznie, gdy wyrocznia jest trwale nieużyteczna, TCR spada poniżej progu krytycznego albo deficyt przekracza limit. Zatrzymuje emisję i likwidacje, a posiadacze stablecoina odbierają zabezpieczenie **w naturze, proporcjonalnie do udziału w całym długu systemu**. Podział nie zależy od żadnej ceny ani od kolejności zgłoszeń, więc nie da się na nim zarobić kosztem innych i nie ma wyścigu do wyjścia.

4.8 Role uczestników

Rola	Co robi	Czego nie zakładamy
Pożyczkobiorca	Deponuje WETH, emituje stablecoina, dopłaca i spłaca	Że zdąży zareagować
Likwidator	Bot zarabiający na premii	Że działa, gdy jest nieopłacalnie
Depozytariusz puli	Kupuje zabezpieczenie z dyskontem, dostaje część odsetek	Że zostanie w puli przed krachem
Wykupujący	Pilnuje podłogi kotwicy	Że działa bez zysku z arbitrażu

5 Historia rewizji

Koncepcja przeszła dwie rewizje. Lista jest jawna z dwóch powodów: każda poprawka zmieniła architekturę albo zakres, a przy publikacji jest dowodem, że projekt był krytycznie przeglądany, a nie tylko rozbudowywany.

5.1 Rewizja przy wersji 0.1

- Dodano model wyroczni z opóźnieniem zamiast ceny znanej natychmiast.
- Porównanie mechanizmów uwzględnia koszt kapitału puli i poślizg na DEX.
- Symulacja ma dwa reżimy kosztów transakcji: L2 i L1 w krachu.
- Dodano wodospad strat, księgę deficytu, model ucieczki z puli i ułamek domknięcia.
- Opisano obie strony kotwicy; wprowadzono minimalną wersję publikowalną.

5.2 Rewizja przy wersji 0.2

Problem w wersji 0.1	Dlaczego to problem	Poprawka w wersji 0.2
Router z funkcją zmiany strategii	To funkcja administracyjna, sprzeczna z decyzją o braku ról	Strategia jako niezmienny parametr wdrożenia; dwa wdrożenia do porównania
Tryb awaryjny z wykupem po ostatniej dobrej cenie, w kolejności zgłoszeń	Arbitraż starej ceny i run przy niedoborze	Podział zabezpieczenia pro rata w naturze
Nieokreślona księgowość długu w aukcji	Od tego zależą TCR, niezmienniki i warunki trybu awaryjnego	Dług w aukcji jako osobna pozycja w księdze deficytu
Brak źródła wynagrodzenia puli	Pula bez dochodu jest fikcją	Część odsetek trafia do puli
Testnet jako dowód ekonomii	Brak prawdziwej płynności i likwidatorów	Główne eksperymenty na forku Base mainnet
Scenariusze z L1 jako scenariusze dla Base	Infrastruktura z 2020 i 2022 to zjawiska L1	Ścieżki cen przenośne; scenariusze infrastrukturalne natywne dla Base
Kalibracja na likwidacjach z L1	Inne opóźnienia; do tego błąd przeżywalności	Kalibracja na protokołach z Base z opisanym błędem przeżywalności
Bufor z empirycznego maksimum	Maksimum rośnie z długością próby	Apetyt na ryzyko jako parametr; kwantyle z blokowego bootstrapu
Analiza kotwicy bez modelu rynku	Cena stablecoina w symulacji nie istnieje	Dynamika kotwicy poza MVP
Premia rosnąca dla małych pozycji	Komplikuje mechanizm	Minimalny dług i rezerwa gazowa
Dowody formalne arytmetyki puli	Arytmetyka nieliniowa, narzędzia się dławią	Dowody prostych własności; resztę pokrywa fuzzing
Ryzyko dopasowania wyników do tezy	Strojenie aż do potwierdzenia	Prerejestracja przed eksperymentami
Bot i panel na testnecie jako dowód	Dowodzą tylko, że bot działa	Bot na forku z odtwarzaną ceną; panel opcjonalny
Kolejka wykupu według ICR	Wymagałaby przeliczania po każdej zmianie ceny	Lista posortowana według NICR z podpowiedziami

6 Decyzje projektowe

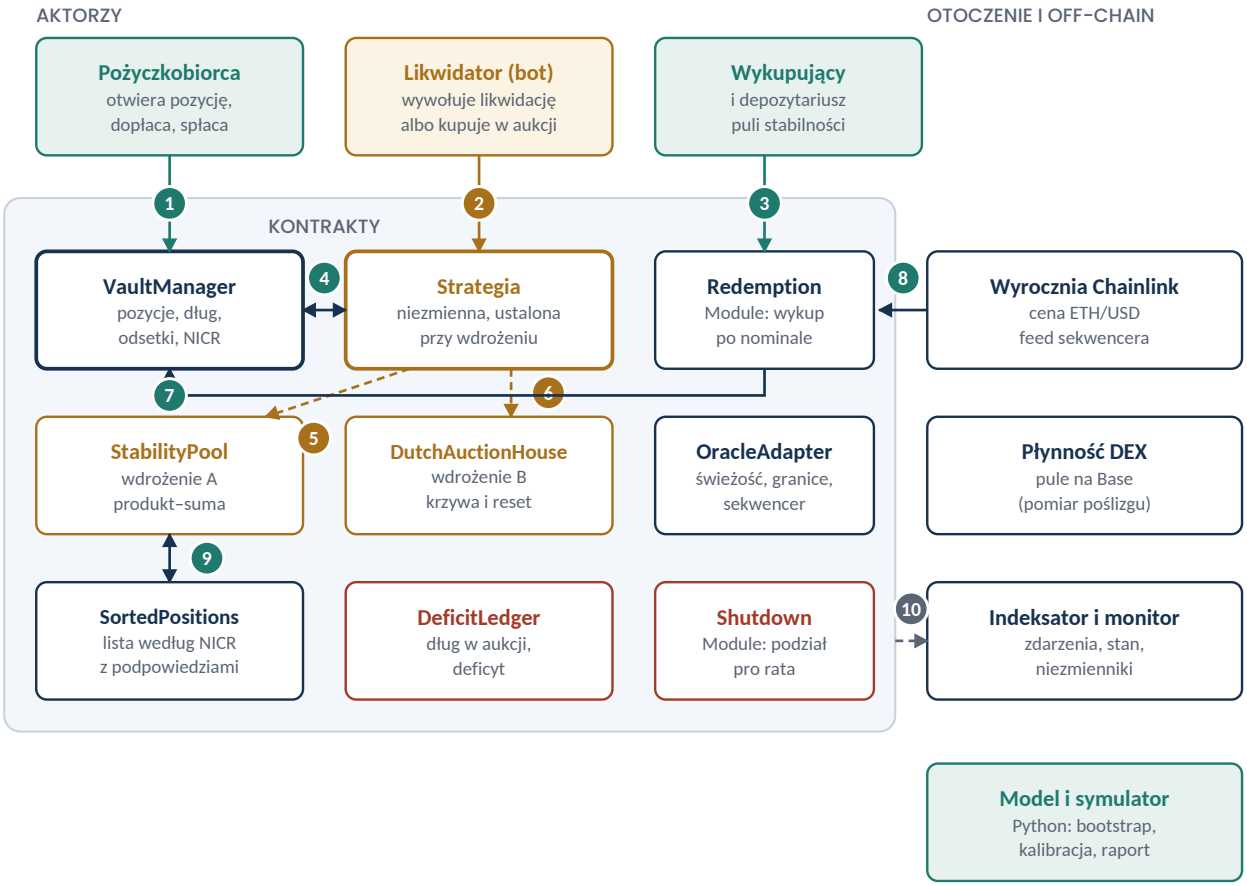
Rejestr w formie skróconej; każda decyzja ma w repozytorium wpis ADR z kontekstem, alternatywami i konsekwencjami. Zmiana decyzji wymaga nowego wpisu, a nie edycji starego.

ADR	Decyzja	Odrzucone alternatywy	Uzasadnienie
01	Kotwica dolarowa	Złoto, koszyk, indeksacja, cena sterowana	Kotwica nie może dokładać problemów
02	Tylko WETH	Koszyk LST, wiele rynków	Dane, prostota, brak ryzyka korelacji
03	Strategia niezmienna per wdrożenie	Router z przełączaniem	Brak administratora
04	Redystrybucja jako szczebel awaryjny	Natychmiastowy deficyt	Utrzymanie wypłacalności po wyczerpaniu puli; oba warianty mierzone
05	Wykup po nominale, kolejka według NICR	Arbitraż rynkowy; kolejka według ICR	Twarda podłoga; kolejność niezależna od ceny
06	Stała stopa	Stopy ustawiane przez użytkowników	Mniej ruchomych części; spójne z NICR
07	Minimalny dług i rezerwa gazowa	Premia rosnąca dla małych pozycji	Rozwiązanie konstrukcyjne
08	Chainlink + feed sekwencera	Własna wyrocznia, TWAP z DEX	Standard; najmniejsze ryzyko
09	Kontrakty niezmienialne	Proxy UUPS	Prostszy model zagrożeń
10	Tryb awaryjny z podziałem pro rata	Wykup po ostatniej dobrej cenie	Brak arbitrażu i runu
11	Fork Base mainnet jako główne środowisko eksperymentów	Testnet; czysta symulacja	Prawdziwa wyrocznia i płynność z momentu krachu
12	Model Pythona + test różnicowy	Symulacja wyłącznie na EVM	Szybkość przemiatania przy zachowaniu zgodności
13	Prerejestracja i blokowy bootstrap	Swobodne strojenie, maksima empiryczne	Wyniki do obrony
14	Rdzeń z własnej specyfikacji	Fork protokołu referencyjnego	Projekt ma być badaniem
15	Arytmetyka WAD i RAY z jawnym kierunkiem zaokrągleń	Domyślne zaokrąglanie w dół	Zaokrąglenia zawsze na korzyść protokołu

7 Architektura

7.1 Widok ogólny

System ma trzy warstwy: **kontrakty** trzymają środki i egzekwują reguły; **otoczenie** — wyrocznia, płynność, sekwencer — jest poza naszą kontrolą, więc jest jawnie modelowane; **warstwa off-chain** to indeksator z monitorem niezmienników oraz model i symulator. Jedno wdrożenie zawiera dokładnie jedną strategię likwidacji; porównanie odbywa się na dwóch wdrożeniach tego samego rdzenia.



Rysunek 2. Architektura jednego wdrożenia. Strzałki przerywane 5 i 6 oznaczają strategie alternatywne: wdrożenie A albo B.

Nr	Przepływ
1	Pożyczkobiorca otwiera pozycję (z minimalnym długiem i rezerwą gazową), dopłaca, pożycza i spłaca.
2	Likwidator wywołuje likwidację w VaultManager; w wdrożeniu z aukcją kupuje zabezpieczenie w DutchAuctionHouse.
3	Wykupujący wymienia stablecoina na zabezpieczenie po nominale; depozytariusz wpłaca do puli.
4	VaultManager przekazuje pozycję strategii zapisanej jako niezmienny parametr wdrożenia.
5	Wdrożenie A: strategią jest pula stabilności (umorzenie długu z depozytów).
6	Wdrożenie B: strategią jest aukcja holenderska. Porównanie robimy na dwóch wdrożeniach.
7	OracleAdapter dostarcza cenę wszystkim modułom; jedno miejsce kontroli świeżości i granic.
8	Wyrocznia Chainlink oraz feed dostępności sekwencera (prawdziwe na forku, mock lokalnie).

Nr	Przepływ
9	Lista pozycji uporządkowana według NICR — podstawa kolejki wykupów i wyboru pozycji.
10	Zdarzenia kontraktów trafiają do indeksatora; monitor sprawdza niezmienniki w ruchu.

7.2 Kontrakty i odpowiedzialności

Kontrakt	Odpowiedzialność	Uwagi
VaultManager	Pozycje, dług znormalizowany, indeks odsetek, ICR i TCR, wejście dla likwidacji i wykupu	Jedyny kontrakt zmieniający stan pozycji
SortedPositions	Lista dwukierunkowa pozycji według NICR, wstawianie z podpowiedziami	Kolejka wykupów i wybór pozycji do likwidacji
StabilityPool	Depozyty, umorzenie długu, podział zabezpieczenia i odsetek	Algorytm produkt–suma; tylko w wdrożeniu A
DutchAuctionHouse	Aukcja malejąca, zakupy częściowe, reset, zakończenie	Tylko w wdrożeniu B
RedemptionModule	Wykup po nominale, opłata z bazową stopą zanikającą w czasie	Podłoga kotwicy
OracleAdapter	Cena, świeżość, granice, sekwencer, karencja, ostatnia dobra cena	Jedyne źródło ceny w systemie
DeficitLedger	Dług w aukcji, deficyt, rezerwa z odsetek, socjalizacja	Strata nie znika po cichu
ShutdownModule	Warunki wejścia, okno spłat, podział pro rata	Reguła, nie decyzja
StableToken	ERC-20 z mennicą i spalaniem, z sufiksem TEST w nazwie wdrożeń testowych	Uprawnienia tylko dla rdzenia

7.3 Granice odpowiedzialności

- **VaultManager nie zna mechaniki likwidacji.** Przekazuje strategii paczkę „dług plus zabezpieczenie” i przyjmuje rozliczenie: ile umorzono, ile zabezpieczenia wyszło, ile deficytu.
- **Strategie nie dotykają pozycji.** Dzięki temu da się je testować w izolacji i podmieniać między wdrożeniami bez zmian w rdzeniu.
- **OracleAdapter nie podejmuje decyzji.** Zwraca cenę i flagę użyteczności; o tym, co wolno, decyduje rdzeń według tabeli w rozdziale 8.8.
- **Każda zmiana stanu emituje zdarzenie** wystarczające do odtworzenia stanu. Indeksator buduje model systemu wyłącznie ze zdarzeń, co jest jednocześnie niezależną weryfikacją księgowości.

8 Specyfikacja techniczna kontraktów

Rozdział jest podstawą dla dokumentu *docs/spec.md* w repozytorium. Opisuje reprezentację liczb, stan, algorytmy i uprawnienia na tyle dokładnie, by implementacja nie wymagała zgadywania.

8.1 Konwencje liczbowe i zaokrąglenia

Kwoty i ceny przechowujemy w skali **WAD** (10^{18}), indeks odsetek w skali **RAY** (10^{27}). Cena z wyroczni Chainlink ma zwykle 8 miejsc po przecinku i jest skalowana do WAD w jednym miejscu: w OracleAdapter. Każda operacja dzielenia ma jawnie określony kierunek zaokrąglenia — zawsze na korzyść protokołu.

Operacja	Kierunek	Uzasadnienie
Dług pozycji z długu znormalizowanego	w górę	Protokół nigdy nie zaniża należności
Dług znormalizowany przy pożyczce	w górę	Pożyczkobiorca nie zyskuje na zaokrągleniu
Dług znormalizowany przy spłacie	w dół	Spłata nie umarza więcej, niż wpłacono
Zabezpieczenie wypłacane użytkownikowi	w dół	Kontrakt nie wypłaca więcej, niż ma
Udziały w puli przy wpłacie i wartość przy wypłacie	w dół	Ochrona pozostałych depozytariuszy
Opłata wykupu	w górę	Opłata nie spada przez zaokrąglenie
Podział pro rata w trybie awaryjnym	w dół	Reszta zostaje w kontrakcie, nie powstaje niedobór
Cena aukcji przy zakupie	w górę	Kupujący nie płaci mniej, niż wynika z krzywej

8.2 Kluczowy stan

Szkic stanu VaultManager

```
struct Position {
    address owner;
    uint128 collateral;      // WETH, WAD
    uint128 normDebt;        // dług znormalizowany: dług = normDebt * index / RAY
    uint128 gasReserve;      // WETH wpłacone przy otwarciu
    uint128 stake;           // udział w redystrybucji
    uint128 snapLColl;       // migawki akumulatorów redystrybucji
    uint128 snapLDebt;
    Status  status;          // Active, Closed, Liquidated, InAuction
}

// VaultManager
uint256 public index;            // RAY, rośnie z odsetkami
uint256 public lastAccrual;      // znacznik czasu ostatniego naliczenia
uint256 public totalNormDebt;
uint256 public totalCollateral;
uint256 public totalStakes;
uint256 public LColl; uint256 public LDebt;   // akumulatory redystrybucji, WAD

// parametry niezmienne (immutable), zapisane przy wdrożeniu
uint256 immutable MCR; uint256 immutable MIN_DEBT; uint256 immutable GAS_RESERVE;
uint256 immutable RATE_PER_SECOND;   // RAY
uint256 immutable POOL_SHARE_BPS;    // udział odsetek dla puli
ILiquidationStrategy immutable strategy;
```

8.3 Odsetki

Przy każdej operacji zmieniającej dług wywoływane jest naliczenie. Nowy indeks to stary indeks pomnożony przez $(1 + r)^{\Delta t}$, gdzie r to stopa na sekundę w skali RAY, a Δt czas od ostatniego naliczenia. Potęgę liczymy szybkim potęgowaniem w skali RAY (wzorzec znany z kontraktów MakerDAO), co daje dokładność niezależną od częstotliwości naliczeń.

Naliczanie odsetek (pseudokod Solidity)

```
function accrue() public {
    uint256 dt = block.timestamp - lastAccrual;
    if (dt == 0) return;
    uint256 newIndex = rmul(index, rpow(RAY + RATE_PER_SECOND, dt));      // RAY
    uint256 interest = mulDivUp(totalNormDebt, newIndex - index, RAY);    // nowy dług w stablecoinie
    index = newIndex; lastAccrual = block.timestamp;
    uint256 toPool = interest * POOL_SHARE_BPS / 10_000;
    token.mint(address(stabilityPool), toPool);      // wdrożenie A; w B do rezerwy
    token.mint(address(deficitLedger), interest - toPool);
    emit InterestAccrued(index, interest, toPool);
}
```

Emisja odsetek zwiększa podaż stablecoina dokładnie o przyrost długu, więc niezmiennik „podaż równa się sumie długów” pozostaje prawdziwy. W wdrożeniu z aukcją, gdzie nie ma puli, całe odsetki trafiają do rezerwy w księdze deficytu.

8.4 Lista pozycji i kolejka wykupu

SortedPositions to lista dwukierunkowa uporządkowana malejąco według NICR = zabezpieczenie / dług znormalizowany. Ponieważ wszystkie pozycje mają to samo zabezpieczenie i tę samą stopę, ruch ceny ani naliczanie odsetek nie zmieniają kolejności. Wstawianie wymaga znalezienia miejsca; robimy to poza łańcuchem i przekazujemy **podpowiedź** w postaci sąsiadów. Kontrakt sprawdza podpowiedź i jeśli jest nieaktualna, przechodzi ograniczoną liczbę kroków po liście.

> **Do weryfikacji testem**
>
> Redystrybucja dodaje pozycjom zabezpieczenie i dług proporcjonalnie do udziałów. Wersja 1 Liquity opiera się na tym, że taka operacja nie psuje uporządkowania. U nas to nie założenie, lecz niezmiennik I13: lista jest posortowana po każdej sekwencji operacji, z uwzględnieniem oczekujących nagród z redystrybucji.

Wykup zaczyna od pozycji o najniższym NICR. Pozycje z ICR poniżej MCR są pomijane — powinny być likwidowane, a nie wykupywane. Każdą pozycję wykupuje się w całości albo częściowo, tak by dług nie spadł poniżej minimum; pozycja wykupiona do zera jest zamykana, a jej nadwyżka zabezpieczenia i rezerwa gazowa czekają na odbiór przez właściciela. Wywołujący podaje maksymalną liczbę iteracji, co ogranicza koszt i chroni przed atakiem przez zapchanie listy.

Opłata wykupu składa się z minimum i **bazowej stopy**, która rośnie proporcjonalnie do wykupionego udziału w podaży i zanika wykładniczo w czasie z ustalonym czasem połowicznego zaniku. Mechanizm odpowiada temu z Liquity v1 i tłumi gwałtowne serie wykupów.

8.5 Pula stabilności: algorytm produkt–suma

Naiwna implementacja musiałaby przy każdej likwidacji aktualizować wszystkie depozyty. Algorytm produkt–suma (opisany w pracy „Scalable Reward Distribution with Compounding Stakes”, Liquity) robi to w czasie stałym, utrzymując dwa globalne akumulatory:

- **P** — iloczyn czynników, przez które zmalały depozyty. Przy umorzeniu długu D z puli o łącznym saldzie T: P ← P · (1 − D/T).

- **S** — suma zysków w zabezpieczeniu na jednostkę depozytu. Przy przejęciu zabezpieczenia C: S ← S + C · P / T.
- **Wartość depozytu** d złożonego przy migawce P_0: d · P / P_0; **zysk w zabezpieczeniu**: d · (S − S_0) / P_0.
- **Epoka** rośnie, gdy pula zostaje wyzerowana (D = T); depozyty z poprzedniej epoki są wtedy zerowe, a ich zyski rozliczane ze starego S.
- **Skala** rośnie, gdy P spadłoby poniżej progu precyzji; P mnożymy wtedy przez stały czynnik, a S przechowujemy osobno dla każdej pary (epoka, skala).

Odsetki trafiające do puli rozliczamy analogicznie, osobnym akumulatorem zysku w stablecoinie. Ten fragment jest historycznie najbardziej podatny na błędy precyzji: dochodzi korekta błędu zaokrągleń przenoszona między operacjami, osobna kampania fuzzingu i niezmiennik I3.

8.6 Redystrybucja

Gdy puli nie ma albo nie wystarcza, niepokryty dług D i zabezpieczenie C rozkładamy na pozostałe pozycje proporcjonalnie do udziałów. Zamiast iterować po pozycjach, zwiększamy globalne akumulatory L_{coll} += C / Σudziałów oraz L_{debt} += D / Σudziałów. Każda pozycja przy najbliższym dotknięciu dolicza sobie udział · (L − $L_{migawka}$) i odświeża migawkę. Wszystkie obliczenia ICR i NICR uwzględniają nagrody oczekujące, więc leniwe naliczanie nie zmienia wyników.

8.7 Aukcja holenderska

Element	Specyfikacja
Start	Dług pozycji przechodzi do księgi jako dług w aukcji; zabezpieczenie do domu aukcyjnego. Cena startowa: cena z wyroczni pomnożona przez bufor (np. 1,2)
Krzywa	Cena(t) = cena startowa · $e^{-k \cdot t}$, liczona w skali WAD; wariant schodkowy jako opcja
Zakup	buy(id, maxCollateral, maxPrice): kupujący podaje maksymalną cenę, co chroni przed wyprzedzeniem; zakupy częściowe dozwolone, jeśli pozostały dług jest zerowy albo większy od minimum
Reset	Gdy minie maksymalny czas aukcji albo cena spadnie poniżej ustalonego ułamka ceny startowej — nowa cena startowa z bieżącej wyroczni
Zakończenie	Dług w aukcji spłacony: nadwyżka zabezpieczenia wraca do właściciela pozycji. Zabezpieczenie wyczerpane przed spłatą: reszta długu trafia do deficytu
Zachęty	Stała premia dla wywołującego start i reset, wypłacana z rezerwy gazowej pozycji

Konstrukcja jest wzorowana na module Clipper z MakerDAO (funkcje kształtu krzywej, maksymalny czas, próg resetu), uproszczona do jednego zabezpieczenia i bez governance.

8.8 OracleAdapter

Logika wyroczni (pseudokod Solidity)

```
function fetchPrice() public returns (uint256 price, bool usable) {
    try sequencerFeed.latestRoundData() returns (uint80, int256 up, uint256 since, uint256, uint80) {
        if (up != 0) return (lastGood, false);                        // sekwencer niedostępny
        if (block.timestamp - since <= GRACE_PERIOD) return (lastGood, false);   // karencja
    } catch { return (lastGood, false); }

    try priceFeed.latestRoundData() returns (uint80, int256 answer, uint256, uint256 updatedAt, uint80) {
        if (answer <= 0) return (lastGood, false);
        if (block.timestamp - updatedAt > STALENESS) return (lastGood, false);
        uint256 p = uint256(answer) * 10 ** (18 - FEED_DECIMALS);    // skala WAD
        if (p < MIN_PRICE || p > MAX_PRICE) return (lastGood, false);
        lastGood = p;
        return (p, true);
    } catch { return (lastGood, false); }
}
```

Operacja	Wymaga użytecznej ceny	Dlaczego
Otwarcie, pożyczka, wypłata zabezpieczenia	tak	Zmniejszają ICR
Spłata, dopłata zabezpieczenia	nie	Użytkownik musi móc ratować pozycję zawsze
Likwidacja, start i reset aukcji	tak	Bez ceny nie wiadomo, czy pozycja jest zagrożona
Wykup	tak	Wykup po złej cenie to transfer wartości
Wpłata i wypłata z puli	nie	Nie zależą od ceny
Odbiór w trybie awaryjnym	nie	Podział pro rata jest niezależny od ceny

8.9 Tryb awaryjny

1. **Wyzwolenie.** Każdy może wywołać sprawdzenie warunków: wyrocznia nieużyteczna dłużej niż ustalony czas, TCR poniżej progu krytycznego albo deficyt powyżej limitu. Spełnienie jednego z nich nieodwracalnie przełącza system.
2. **Zamrożenie.** Koniec emisji, likwidacji, wykupów i startów aukcji. Trwające aukcje kończą się według swoich reguł albo są zamykane z przeniesieniem długu do deficytu.
3. **Okno spłat.** Przez ustalony czas pożyczkobiorcy mogą spłacić dług i odebrać zabezpieczenie; depozytariusze mogą wycofać środki z puli.
4. **Podział.** Po zamknięciu okna całe pozostałe zabezpieczenie trafia do wspólnej puli, a każdy posiadacz stablecoina odbiera część proporcjonalną do swojego udziału w niespłaconej podaży.

Koszt tego projektu jest jawny: pożyczkobiorca, który nie zareaguje w oknie spłat, traci nadwyżkę zabezpieczenia na rzecz posiadaczy stablecoina. Alternatywą jest rozliczenie pozycji po cenie z chwili wyzwolenia, gdy przyczyną nie była awaria wyroczni — to otwarte pytanie z rozdziału 23.

8.10 Zdarzenia

Zdarzenie	Pola kluczowe
PositionOpened / Adjusted / Closed	id, właściciel, zabezpieczenie, dług znormalizowany, rezerwa
InterestAccrued	nowy indeks, przyrost długu, część dla puli
Liquidated	id, dług, zabezpieczenie, rekompensata, wykonawca
PoolOffset	umorzony dług, przejęte zabezpieczenie, P, S, epoka, skala

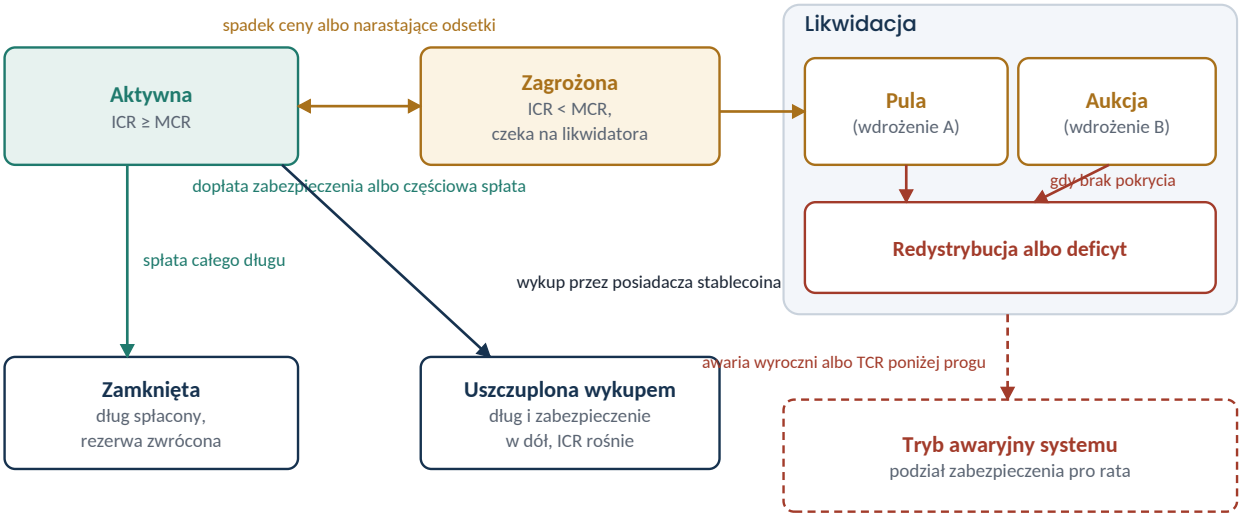
Zdarzenie	Pola kluczowe
Redistributed	dług, zabezpieczenie, nowe L_{coll} i L_{debt}
AuctionStarted / Bought / Reset / Finished	id aukcji, cena, ilości, dług pozostały, deficyt
Redeemed	kwota, zabezpieczenie wydane, opłata, liczba dotkniętych pozycji
DeficitRecorded / Socialized	kwota, źródło
ShutdownTriggered / Claimed	przyczyna, migawki; odbiorca i kwota

Zasada: stan każdego kontraktu musi dać się odtworzyć wyłącznie ze zdarzeń. Indeksator robi to niezależnie od kontraktów i porównuje wynik z odczytami stanu — rozbieżność to alarm.

8.11 Macierz uprawnień

Funkcja	Kto może wywołać
open, borrow, withdrawCollateral, close	Właściciel pozycji
addCollateral, repay	Każdy (na rzecz dowolnej pozycji)
accrue, liquidate, startAuction, resetAuction, checkShutdown	Każdy
redeem, buy (aukcja), deposit i withdraw (pula), claim	Każdy, we własnym imieniu
offset (pula), mint i burn (token), recordDeficit	Wyłącznie wskazane kontrakty rdzenia
Zmiana parametrów, strategii, wyroczni	Nikt — wszystko niezmienne od wdrożenia

9 Cykl życia pozycji



Rysunek 3. Stany pozycji i przejścia.

Przejście	Kto	Warunek	Skutek
Brak → Aktywna	Pożyczkobiorca	Dług ≥ minimum, ICR ≥ MCR, cena użyteczna, rezerwa wpłacona	Wpis na liście według NICR
Aktywna → Zagrożona	Rynek	Cena albo odsetki obniżają ICR poniżej MCR	Pozycja likwidowalna
Zagrożona → Aktywna	Pożyczkobiorca	Dopłata lub częściowa spłata	ICR powyżej progu
Zagrożona → Likwidacja	Każdy	ICR < MCR, cena użyteczna, sekwencer po karencji	Wdrożenie A: umorzenie z puli; B: start aukcji
Likwidacja → Redystrybucja	System	Pula pusta lub niewystarczająca	Akumulatory L rosną
Aktywna → Uszczuplona wykupem	Wykupujący	Najniższy NICR wśród pozycji z ICR ≥ MCR	Dług i zabezpieczenie maleją
Aktywna → Zamknięta	Właściciel	Spłata całości	Zwrot zabezpieczenia i rezerwy
Dowolny → Tryb awaryjny	Każdy (sprawdzenie warunków)	Wyrocznia, TCR albo deficyt	Okno spłat, potem podział pro rata

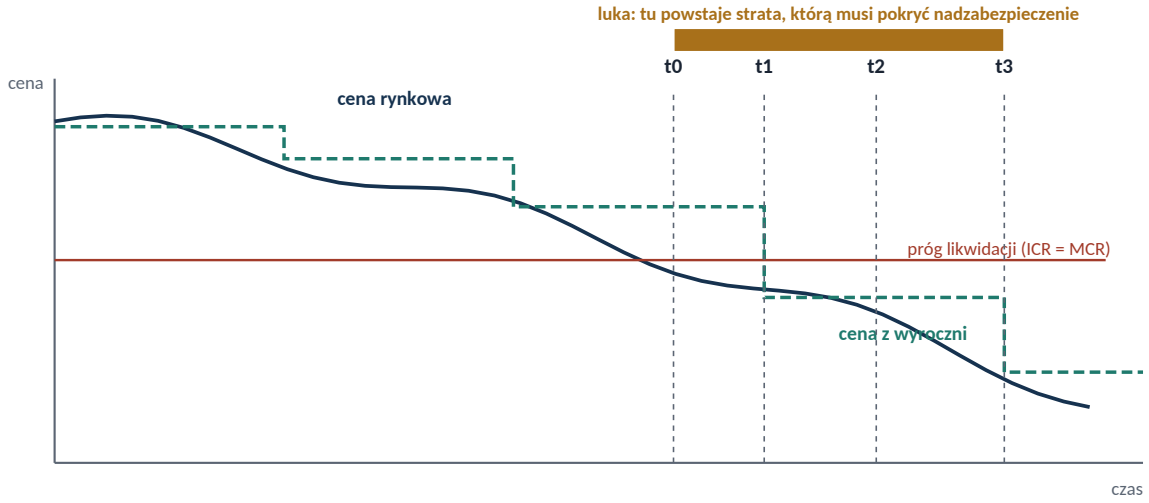
9.1 Scenariusze

Scenariusz	Przebieg	Co mierzymy
A. Zwykła likwidacja	Wyrocznia aktualizuje się, bot wykrywa pozycję, sprawdza opłacalność, likwiduje; nadwyżka wraca do właściciela	Opóźnienie, oddana wartość
B. Gwałtowny spadek	Cena wyprzedza wyrocznię; po aktualizacji zabezpieczenie jest warte mniej niż dług	Luka, dług nieściągalny
C. Brak chętnych	Aukcja spada, ale rynek leci dalej; resety	Czas do rozliczenia, liczba resetów
D. Przerwa sekwencera	Brak transakcji, po powrocie karencja, potem fala likwidacji	Koszt karencji kontra koszt likwidacji na starych danych

Scenariusz	Przebieg	Co mierzymy
E. Wyczerpanie puli	Kilka likwidacji pod rząd wyczerpuje pulę, wchodzi redystrybucja	Przeniesienie straty na pożyczkobiorców

10 Wyrocznia i budżet opóźnień

Protokół nigdy nie widzi ceny rynkowej, tylko cenę zapisaną przez wyrocznię, z opóźnieniem wynikającym z jej konfiguracji. W tej szczelinie powstaje większość długu nieściągalnego.



Rysunek 4. t0 — cena rynkowa przebija próg; t1 — wyrocznia pokazuje nową cenę; t2 — bot wysyła transakcję; t3 — likwidacja w bloku. Schemat.

10.1 Składniki opóźnienia

Składnik	Skąd wynika	Wpływ	Jak mierzymy
Próg odchylenia	Aktualizacja po ruchu o ustalony procent	Nie	Historia feedu na Base
Heartbeat	Wymuszona aktualizacja po czasie	Nie	Historia feedu na Base
Karencja sekwencera	Blokada po powrocie sekwencera	Tak	Parametr; symulacja
Cykl odpytania bota	Częstotliwość sprawdzania stanu	Tak	Konfiguracja bota
Włączenie do bloku	Czas bloku i kolejkowanie	Częściowo	Pomiar na forku i testnecie
Konkurencja botów	Inni likwidatorzy	Nie	Kalibracja na zdarzeniach z Base

10.2 Pierwsze przybliżenie bufora

Przy ruchu losowym o rocznej zmienności σ niekorzystny ruch w czasie T na poziomie trzech odchyleń wynosi około 3·σ·√(T/rok):

Zmienność roczna	T = 5 min	T = 15 min	T = 1 h	T = 4 h
60%	0,56%	0,96%	1,92%	3,85%
100%	0,93%	1,60%	3,21%	6,41%
150%	1,39%	2,40%	4,81%	9,62%

Dlaczego to tylko dolne ograniczenie

Rozkłady zwrotów mają grube ogony i skupiają zmienność w czasie. W krachu z marca 2020 cena ETH spadła o kilkadziesiąt procent w ciągu doby — rząd wielkości więcej, niż przewiduje model normalny. Właściwy bufor wyprowadzamy z symulacji opisanej niżej.

10.3 Bufor z apetytu na ryzyko

Wersja 0.1 zakładała bufor z „empirycznego maksimum ruchu w oknie opóźnienia". To nie daje się obronić: maksimum rośnie z długością próby. W wersji 0.2 bufor wynika z dwóch jawnych elementów:

1. **Apetyt na ryzyko** — parametr zapisany w prerejestracji, np. akceptowalne prawdopodobieństwo, że pozycja wygeneruje dług nieściągalny w ciągu roku.
2. **Rozkład ruchu w oknie opóźnienia** — uzyskany z wielu ścieżek wygenerowanych **blokowym bootstrapem** historycznych zwrotów. Losowanie bloków, a nie pojedynczych zwrotów, zachowuje skupianie się zmienności, więc gwałtowne epizody pozostają gwałtowne.

Bufor to kwantyl tego rozkładu odpowiadający apetytowi na ryzyko, z przedziałem ufności z powtórzeń bootstrapu. Wynik raportujemy jako krzywą „bufor w funkcji apetytu na ryzyko", a nie jedną liczbę.

10.4 Składniki minimalnego wskaźnika zabezpieczenia

Składnik	Co pokrywa	Skąd wartość
Premia likwidatora	Wynagrodzenie bota	Warunek opłacalności (11.4)
Bufor luki	Ruch ceny między t0 a t3	Kwantyl z bootstrapu dla apetytu na ryzyko
Bufor poślizgu	Koszt sprzedaży zabezpieczenia	Pomiary na forku i model płynności
Margines modelu	Niepewność samego modelu	Decyzja jawna w raporcie

10.5 Obrona przed złą ceną

- **Świeżość i granice wartości** — odczyt stary lub poza przedziałem jest nieużyteczny.
- **Wywołania w bloku try/catch** — awaria feedu nie blokuje kontraktu, tylko zwraca flagę.
- **Feed sekwencera i karencja** — brak likwidacji na cenach z okresu, w którym użytkownicy nie mogli działać.
- **Degradacja zamiast awarii** — spłata i dopłata są zawsze możliwe.
- **Tryb awaryjny** — trwała awaria wyroczni kończy się uporządkowanym zamknięciem.

11 Mechanizmy likwidacji

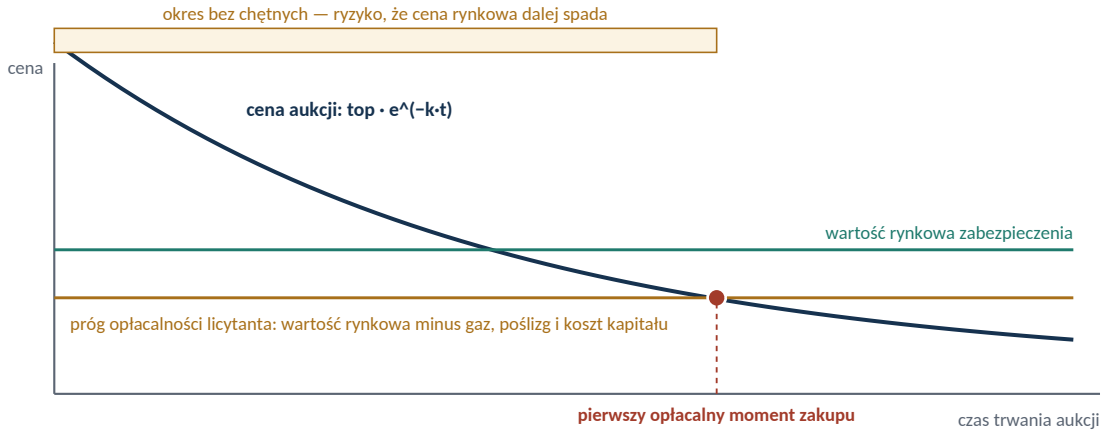
Oba mechanizmy rozwiązują ten sam problem, ale różnie odpowiadają na pytanie, **skąd w chwili likwidacji bierze się kapitał**. To przesądza o szybkości, koszcie, podziale ryzyka i granicach stosowalności.

11.1 Pula stabilności (wdrożenie A)

Kapitał jest wpłacony z góry. Pula umarza dług zagrożonej pozycji i przejmuje jej zabezpieczenie z dyskontem; wywołujący dostaje rezerwę gazową pozycji. Likwidacja jest natychmiastowa i nie wymaga rynku. Za gotowość kapitału pula dostaje część odsetek. Ryzyka: pula za mała, ucieczka depozytariuszy przed krachem, poślizg przeniesiony na depozytariuszy upłynniających zabezpieczenie.

11.2 Aukcja holenderska (wdrożenie B)

Kapitał pozyskiwany jest na żądanie. Zabezpieczenie trafia na aukcję z malejącą ceną; licytant zwykle finansuje zakup pożyczką błyskawiczną i natychmiast sprzedaje zabezpieczenie na DEX, więc o jego decyzji rozstrzygają poślizg i koszty. Ryzyka: brak chętnych, krzywa goniąca spadający rynek, resety, większa część wartości oddana na zewnątrz.



Rysunek 5. Krzywa ceny aukcji i pierwszy opłacalny moment zakupu.

11.3 Redystrybucja i deficyt

Gdy żaden mechanizm nie pokrywa długu, strata trafia do redystrybucji albo do księgi deficytu. Oba warianty są zaimplementowane i mierzone; wybór domyślnego jest otwartym pytaniem.

11.4 Opłacalność likwidacji

Dla pozycji o wartości zabezpieczenia V, premii p, poślizgu s i koszcie transakcji K likwidacja jest opłacalna, gdy:

V · (p − s) ≥ K, czyli **V ≥ K / (p − s)**

Koszt transakcji	Premia 5%, poślizg 0,5%	Premia 10%, poślizg 2%
0,03 USD (L2, normalnie)	0,67 USD	0,38 USD
2 USD (L2, zatłoczenie)	44,44 USD	25,00 USD
30 USD (L1, krach)	667 USD	375 USD

Koszt transakcji	Premia 5%, poślizg 0,5%	Premia 10%, poślizg 2%
150 USD (L1, ekstremum)	3 333 USD	1 875 USD

> **Co z tego wynika**
>
> Na L2 bariera kosztu transakcji praktycznie znika — wiązaniem staje się **poślizg**, a przy dużych pozycjach rośnie on nieliniowo. Minimalny dług pozycji ustalamy więc nie z kosztu gazu, tylko z zapasu: tak, by pozostał opłacalny nawet w reżimie L1 w krachu. Rezerwa gazowa pokrywa koszt wywołania niezależnie od premii.

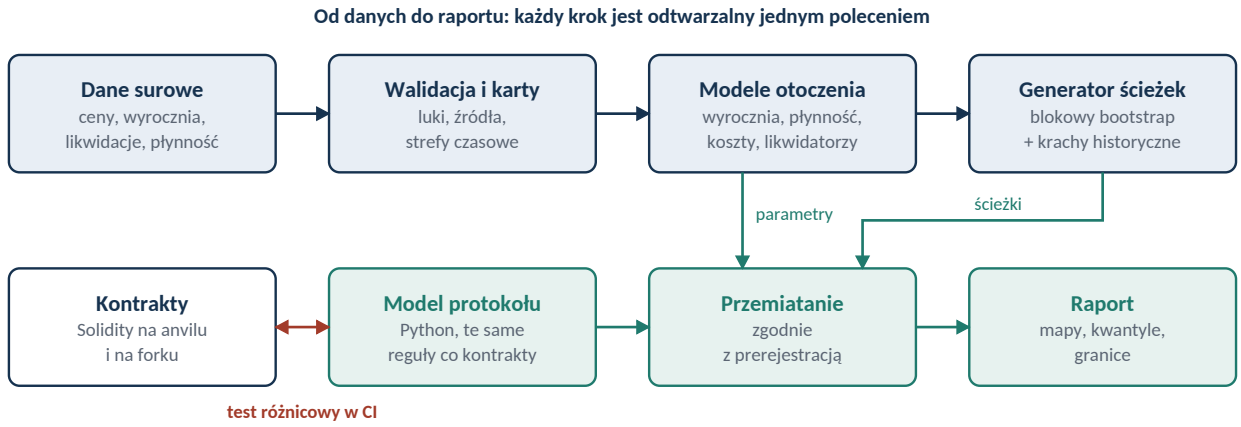
11.5 Osie porównania

Oś	Pula stabilności	Aukcja holenderska
Źródło kapitału	Wpłacony z góry	Na żądanie
Zależność od płynności	Odroczona, po stronie depozytariuszy	Natychmiastowa, po stronie licytanta
Czas rozliczenia	Jedna transakcja	Sekundy do minut, z resetami
Koszt stały	Część odsetek dla puli	Brak
Ryzyko braku wykonania	Pusta pula	Brak chętnych
Kto ponosi stratę w ogonie	Depozytariusze, potem pożyczkobiorcy	Protokół przez przecenę

Hipoteza robocza, zapisana w prerejestracji: pula wygrywa w gwałtownych, krótkich krachach, aukcja w długich osuwiskach. Jeśli symulacja ją obali, raport opisze to równie dokładnie.

12 Model protokołu i symulator

Zasada naczelna: **symulator nie zgaduje tego, co można zmierzyć**. Ceny pochodzą z danych, wyrocznia odtwarza rzeczywistą konfigurację feedu na Base, płynność jest mierzona na forkach, a likwidatorzy są kalibrowani na prawdziwych zdarzeniach.



Rysunek 6. Potok od danych do raportu i miejsce testu różnicowego.

12.1 Dwie implementacje tych samych reguł

Kontrakty na EVM są wiarygodne, ale wolne: tysiące scenariuszy z setkami pozycji to godziny. Dlatego reguły protokołu implementujemy drugi raz, w Pythonie, jako szybki model używany do przemiatania parametrów. Ryzyko rozjazdu dwóch implementacji neutralizuje **test różnicowy**: losowy scenariusz uruchamiany na obu i porównanie sald, długu, deficytu i stanu listy. Rozbieżność ponad tolerancję zaokrągleń zatrzymuje CI.

12.2 Moduły

Moduł	Zadanie	Źródło parametrów
model/protocol	Reguły rdzenia: pozycje, odsetki, NICR, wykup, pula, aukcja, redystrybucja, deficyt, tryb awaryjny	Specyfikacja; zgodność z kontraktami przez test różnicowy
model/oracle	Zamiana ceny rynkowej na cenę widzianą przez protokół: próg odchylenia, heartbeat, przerwy sekwencera	Historia feedu na Base
model/liquidity	Poślizg jako funkcja wielkości sprzedaży i stanu rynku	Pomiary kwotowań na forkach
model/costs	Koszt transakcji w reżimach L2 i L1 w krachu	Historia opłat
agents/borrowers	Rozkład wielkości i ICR pozycji; reakcje na ryzyko	Rozkłady z protokołów na Base
agents/liquidators	Opóźnienie, próg opłacalności, konkurencja	Kalibracja (rozdz. 13)
agents/depositors	Wpłaty, wypłaty, ucieczka przed krachem	Parametr wrażliwości w przemiataniu
paths	Ścieżki cen: historyczne i blokowy bootstrap	Dane cenowe
engine	Pętla zdarzeń z krokiem równym aktualizacji wyroczni lub blokowi	—
experiments	Przemiatanie, zapis wyników z ziarnem losowym, wykresy	Preprejestracja

12.3 Pętla symulacji

Szkic pętli (Python)

```
for t, market_price in path:                      # krok: minuta albo blok
    oracle.observe(t, market_price)               # aktualizacja tylko przy progu lub heartbeacie
    protocol.accrue(t)
    for agent in shuffle(borrowers):  agent.act(t, protocol, oracle)
    for agent in shuffle(depositors): agent.act(t, protocol, oracle)
    candidates = protocol.liquidatable(oracle.price)          # na cenie z wyroczni, nie rynkowej
    for liq in liquidators:
        for pos in liq.select(candidates, t):                 # opóźnienie i próg opłacalności
            cost = costs.tx(t) + liquidity.slippage(pos.collateral, t)
            if liq.profitable(pos, oracle.price, market_price, cost):
                protocol.liquidate(pos, by=liq, t=t)
    metrics.record(t, protocol, market_price)
```

12.4 Czego model nie udowadnia

- Agenci to model, nie ludzie; kalibracja ogranicza, ale nie usuwa tej różnicy.
- Płynność w modelu jest funkcją, a w krachu bywa zjawiskiem — księgi zleceń potrafią zniknąć.
- Historia pokazuje, co już się zdarzyło, nie granicę możliwego.
- Wyniki są warunkowe; kod i konfiguracje są publiczne, by każdy mógł podmienić założenia.

13 Dane i kalibracja

13.1 Zestawy danych

Zestaw	Źródło	Format i zakres	Pułapki
Ceny ETH/USD	Publiczne API giełd (świece minutowe), druga giełda do kontroli	Parquet; kilka lat	Rozjazdy giełd w panice, luki, strefy czasowe
Historia wyroczni na Base	Zdarzenia agregatora Chainlink przez RPC archiwalne lub Dune	Parquet: czas, wartość, runda	To prawdziwe wejście protokołu, nie zastępować ceną giełdową
Historia sekwencera	Zdarzenia feedu dostępności na Base	Okresy niedostępności	Rzadkie zdarzenia; mała próba
Zdarzenia likwidacji	Protokoły pożyczkowe na Base (np. Aave v3, Moonwell)	Parquet: pozycja, czas, wielkość, wykonawca	Błąd przeżywalności (13.3)
Płynność	Kwotowania sprzedaży WETH na forkach w wybranych blokach	Tabela: blok, wielkość, cena wykonania	Migawka stanu, nie dynamika
Koszty transakcji	Historia bloków L2 i L1	Szereg czasowy	Na L2 koszt zależy też od publikacji danych na L1

> **Karta zestawu danych**
> Każdy zestaw ma plik karty: źródło, zakres czasu, data pobrania, skrypt pobierający, wykryte luki, sposób ich uzupełnienia i suma kontrolna pliku. Dane surowe nie są edytowane ręcznie; wszystkie przekształcenia są kodem.

13.2 Pomiar płynności na forku

Historycznych danych o głębokości rynku zwykle nie ma — ale na Base są historyczne **stany** pul DEX. Uruchamiamy anvil jako fork Base mainnet w wybranym bloku i odpytujemy kontrakty kwotujące największych pul WETH/USDC o cenę sprzedaży różnych ilości WETH. Wywołania są statyczne, nie zmieniają stanu. Powtarzając pomiar w blokach z okresów spokoju i krachu, dostajemy empiryczną zależność poślizgu od wielkości sprzedaży w różnych warunkach rynkowych.

13.3 Kalibracja likwidatorów i błąd przeżywalności

1. Dla każdej aktualizacji wyroczni odtwarzamy stan pozycji w protokole referencyjnym i wyznaczamy te, które właśnie przekroczyły próg.
2. Dla pozycji zlikwidowanych liczymy opóźnienie od tej aktualizacji do transakcji likwidacji.
3. Dopasowujemy parametry agenta likwidatora — opóźnienie reakcji, próg opłacalności, liczbę konkurentów — tak, by rozkład opóźnień w symulacji odpowiadał danym.
4. Osobno liczymy pozycje, które przekroczyły próg i **nie** zostały zlikwidowane w danym oknie. Ignorowanie ich zaniża opóźnienia — to błąd przeżywalności, który raport opisuje i szacuje.

Odtworzenie stanu pozycji jest najbardziej pracochłonnym krokiem projektu. Wariant tańszy: gotowe tabele zdarzeń z publicznych zbiorów danych i rekonstrukcja tylko dla okien wokół gwałtownych ruchów ceny.

13.4 Scenariusze

ID	Scenariusz	Wejście	Cel pomiaru
S1	Spokojny rynek	Ścieżki z bootstrapu, niska zmienność	Koszt bieżący, wynagrodzenie puli
S2	Krach dobowy (ścieżka z marca 2020)	Ścieżka historyczna, infrastruktura Base	Luka, dług nieściągalny
S3	Krach natywny dla Base (sierpień 2024)	Fork historyczny: wyrocznia i płynność z tamtych bloków	Realne otoczenie
S4	Długie osuwisko	Ścieżki z bootstrapu o długich spadkach	Wyczerpanie puli, resety aukcji
S5	Szok z odbiciem	Ścieżki syntetyczne	Niepotrzebne likwidacje
S6	Zamrożona wyrocznia	Wstrzymane aktualizacje	Reguły świeżości, tryb awaryjny
S7	Przerwa sekwencera	Okno niedostępności i karencja	Koszt karencji
S8	Cienka płynność	Poślizg z bloków krachu, skalowany	Granica działania aukcji
S9	Ucieczka z puli	Wysoka wrażliwość depozytariuszy	Odporność wdrożenia A

14 Metoda badawcza i metryki

14.1 Prerejestracja

Przed pierwszym przemiataniem w repozytorium zostaje zapisany i zatwierdzony plik *docs/preregistration.md*. Zawiera on hipotezy, definicje metryk, apetyt na ryzyko, listę scenariuszy, siatkę parametrów i reguły rozstrzygania („hipoteza H1 jest potwierdzona, jeśli…"). Zmiany po tym momencie są dozwolone, ale tylko jako nowe, datowane wpisy z uzasadnieniem — historia repozytorium pokazuje, co było planem, a co korektą. Szablon zawiera dodatek E.

14.2 Metryki

Metryka	Definicja
Dług nieściągalny	Część długu niepokryta przez zabezpieczenie, pulę ani redystrybucję
Częstość zdarzeń z długiem nieściągalnym	Odsetek pozycji, które go wygenerowały, w przeliczeniu na rok
Czas pod progiem	Suma czasu, przez który pozycje są likwidowalne, a nie zlikwidowane
Wartość oddana na zewnątrz	Premie, dyskonto aukcji i poślizg
Koszt kapitału puli	Odsetki wypłacone depozytariuszom
Całkowity koszt systemu	Wartość oddana + koszt kapitału puli + deficyt — główna metryka porównania
Niepotrzebne likwidacje	Pozycje zlikwidowane, które po odbiciu byłyby zdrowe
Przeniesienie straty	Część straty pokryta redystrybucją, czyli przez innych pożyczkobiorców

14.3 Raportowanie

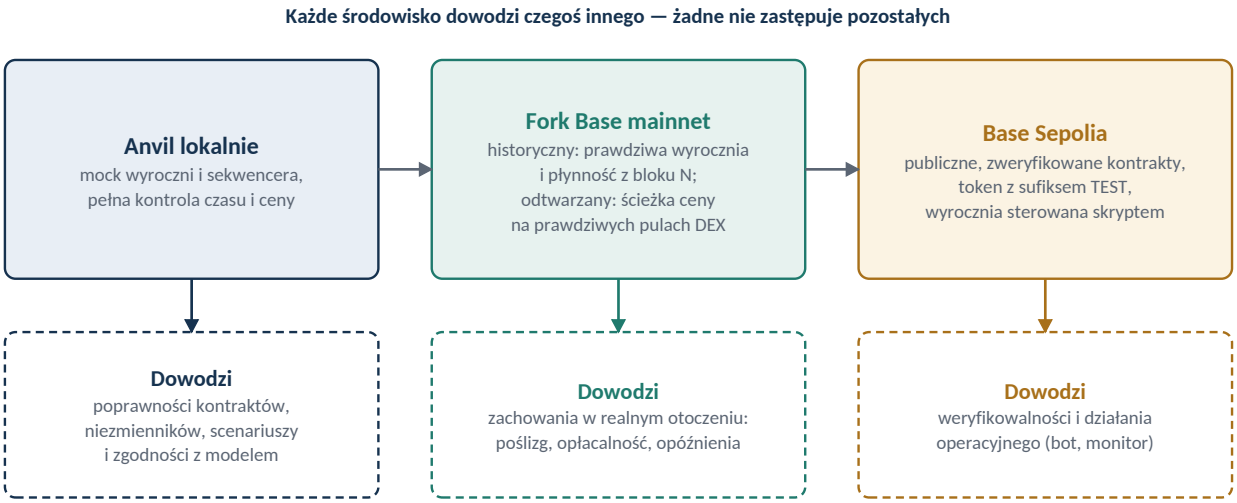
- **Z warunkami**: scenariusz, parametry, wersja danych, ziarno losowe.
- **Mapy, nie punkty**: obszar bezpieczny i jego granica.
- **Z przedziałami**: rozrzut z powtórzeń bootstrapu, nie tylko średnia.
- **Z porażkami**: scenariusze przegrane opisane równie dokładnie jak wygrane.
- **Odtwarzalnie**: każdy wykres powstaje jednym poleceniem.

14.4 Szkielet raportu końcowego

1. Pytanie badawcze, model i prerejestracja.
2. Dane i kalibracja: skąd parametry agentów i jak dobrze odtwarzają rzeczywistość.
3. Wyniki dla wdrożenia A w scenariuszach S1–S9.
4. Wyniki dla wdrożenia B w tych samych scenariuszach.
5. Porównanie, weryfikacja hipotez i granice stosowalności.
6. Wyprowadzone parametry rozbite na składniki, jako funkcje apetytu na ryzyko.
7. Ograniczenia i co należałoby zrobić w wersji produkcyjnej.

15 Środowiska i wdrożenie testowe

Stablecoin z tego projektu nigdy nie będzie oferowany jako pieniądz. Wdrożenie testowe ma za to pokazać trzy rzeczy: że kontrakty działają poprawnie, że ich ekonomia wytrzymuje realne otoczenie i że całość da się uruchomić, zweryfikować i obsługiwać jak prawdziwy system. Każdej z nich dowodzi inne środowisko.



Rysunek 7. Trzy środowiska i to, czego każde dowodzi.

Środowisko	Wyrocznia	Płynność	Czas	Do czego
Anvil lokalnie	Mock z ręcznie ustawianą ceną	Brak lub mock	Pełna kontrola	Praca codzienna, testy, test różnicowy
Fork historyczny	Prawdziwy feed Chainlink z bloku N	Prawdziwe pule z bloku N	Od bloku N, sterowany	Pomiar płynności, scenariusz S3
Fork odtwarzany	ReplayAggregator z historyczną ścieżką	Prawdziwe pule (poślizg względny)	Sterowany	Scenariusze S2, S4–S9 w realnym otoczeniu
Base Sepolia	ReplayAggregator sterowany skryptem	Brak realnej	Rzeczywisty	Publiczna wizytówka, bot, monitor

15.1 Anvil lokalnie

Lokalny węzeł z mockami: **MockAggregator** (cena ustawiana z testu lub skryptu) i **MockSequencerFeed** (stan „działa” albo „przerwa” z datą zmiany). W testach Foundry czas i blok przesuwamy przez *vm.warp* i *vm.roll*; w skryptach przez metody RPC anvila. Wdrożenie jest deterministyczne (CREATE2 z ustalonym salt), więc adresy są stałe między uruchomieniami, co upraszcza skrypty i panel.

Uruchomienie lokalne

```
anvil --chain-id 31337 --block-time 2                       # bloki co 2 s, jak na Base
forge script script/Deploy.s.sol:Deploy --rpc-url http://127.0.0.1:8545 \
      --private-key $ANVIL_KEY_0 --broadcast                # klucz testowy anvila, tylko lokalnie
cast rpc evm_increaseTime 3600 && cast rpc anvil_mine 1     # przesunięcie czasu o godzinę
```

15.2 Fork Base mainnet: tryb historyczny

Anvil uruchomiony jako fork Base mainnet w wybranym bloku ma prawdziwy stan z tamtej chwili: feed Chainlink, feed sekwencera, predeploy WETH i pule DEX. Na takim forku wdrażamy nasze kontrakty wskazujące prawdziwe adresy feedów, zasilamy konta testowe przez ustawienie salda ETH i wpłatę do WETH, a następnie mierzymy poślizg statycznymi wywołaniami kontraktów kwotujących. Do tego potrzebny jest dostawca RPC z dostępem do stanu archiwalnego; anvil buforuje pobrany stan lokalnie, więc kolejne uruchomienia tego samego bloku są szybkie.

Fork historyczny (adresy feedów i pul pochodzą z pliku konfiguracyjnego)

```
anvil --fork-url $BASE_ARCHIVE_RPC --fork-block-number $BLOCK --chain-id 8453
cast rpc anvil_setBalance $TEST_ACCOUNT 0x56BC75E2D63100000           # 100 ETH na koncie testowym
cast send 0x4200000000000000000000000000000000000006 "deposit()" \
     --value 50ether --from $TEST_ACCOUNT --unlocked                  # WETH (predeploy OP Stack)
forge script script/DeployFork.s.sol --rpc-url http://127.0.0.1:8545 --unlocked \
     --sender $TEST_ACCOUNT --broadcast                               # adresy feedów z konfiguracji
python -m experiments.liquidity --rpc http://127.0.0.1:8545 --block $BLOCK
```

15.3 Fork Base mainnet: tryb odtwarzany

Tryb historyczny ma jedno ograniczenie: cena w nim jest taka, jaka była w bloku N, i nie da się jej poprowadzić ścieżką z innego okresu. W trybie odtwarzanym OracleAdapter wskazuje nasz **ReplayAggregator** — kontrakt z interfejsem feedu Chainlink, do którego skrypt wpisuje kolejne punkty historycznej ścieżki, z prawdziwą logiką progu odchylenia i heartbeatu. Pule DEX zostają prawdziwe. Ponieważ ich cena nie podąża za odtwarzaną ścieżką, poślizg liczymy względnie, jako procent od ceny środkowej puli przy danej wielkości sprzedaży, a nie w cenach bezwzględnych.

15.4 Base Sepolia: procedura krok po kroku

1. **Konto wdrożeniowe.** Nowy klucz używany wyłącznie do tego projektu, zaimportowany do zaszyfrowanego magazynu kluczy Foundry: *cast wallet import redoubt-deployer --interactive*. Testowe ETH z faucetu Base Sepolia. Osobne konta dla bota i dla skryptu sterującego ceną.
2. **Konfiguracja.** W *foundry.toml* sekcje z adresem RPC i kluczem API eksploratora; w pliku konfiguracji wdrożenia — adresy feedów, WETH i parametry protokołu. Żadnych kluczy prywatnych w plikach ani w zmiennych środowiskowych.
3. **Wyrocznia.** Na testnecie nie ma prawdziwego rynku, więc publiczne wdrożenie korzysta z ReplayAggregator sterowanego skryptem i z MockSequencerFeed. To uczciwe, jeśli jest jawne: README i opis kontraktu mówią wprost, że cenę ustawia operator demonstracji. Opcjonalnie drugie wdrożenie czyta testnetowy feed Chainlink, jeśli jest dostępny dla ETH/USD na Base Sepolia — do sprawdzenia w dokumentacji Chainlink przed wdrożeniem.
4. **Wdrożenie.** *forge script script/Deploy.s.sol:Deploy --rpc-url base_sepolia --account redoubt-deployer --broadcast --verify*. Skrypt wdraża oba warianty (A i B) z tym samym rdzeniem.
5. **Manifest.** Skrypt zapisuje *deployments/84532.json*: adresy, numer bloku, hash commita, wersję kompilatora i parametry. Manifest trafia do repozytorium i jest źródłem adresów dla bota, indeksatora i README.
6. **Weryfikacja kodu.** Automatyczna przez flagę *--verify*; w razie niepowodzenia ręcznie przez *forge verify-contract*. Zweryfikowany kod źródłowy w eksploratorze to warunek publikacji.
7. **Test dymny.** Skrypt *SmokeTest.s.sol* wykonuje na świeżym wdrożeniu pełną ścieżkę: otwarcie pozycji, naliczenie odsetek, mały wykup, obniżenie ceny przez ReplayAggregator, likwidację, sprawdzenie zdarzeń i funkcji widoku niezmienników.

8. **Uruchomienie bota i monitora.** Bot czyta adresy z manifestu, indeksator startuje od bloku wdrożenia. Monitor zgłasza każde naruszenie niezmiennika.
9. **Demonstracja krachu.** Skrypt sterujący ceną odtwarza wybraną historyczną ścieżkę w przyspieszonym tempie; widać likwidacje wykonywane przez bota bez udziału człowieka.
10. **Nowa wersja.** Kontrakty są niezmienialne, więc poprawka oznacza nowe wdrożenie i nowy manifest; poprzednie adresy są oznaczone w README jako wycofane.

deployments/84532.json — przykład manifestu (wartości poglądowe)

```
{
  "chainId": 84532,
  "commit": "a1b2c3d",
  "solc": "0.8.x",
  "deployedAtBlock": 12345678,
  "variants": {
    "A_stability_pool": { "VaultManager": "0x...", "StabilityPool": "0x...", "SortedPositions": "0x...",
                          "RedemptionModule": "0x...", "DeficitLedger": "0x...", "ShutdownModule": "0x..." },
    "B_dutch_auction":  { "VaultManager": "0x...", "DutchAuctionHouse": "0x...", "...": "..." }
  },
  "shared": { "StableToken_A": "0x...", "StableToken_B": "0x...", "OracleAdapter": "0x...",
              "ReplayAggregator": "0x...", "MockSequencerFeed": "0x...", "WETH": "0x4200...0006" },
  "params": { "MCR_bps": 13500, "MIN_DEBT": "200e18", "GAS_RESERVE": "0.002e18", "RATE_bps_year": 300 }
}
```

15.5 Klucze i sekrety

- Klucze prywatne wyłącznie w zaszyfrowanym magazynie kluczy Foundry; w plikach *.env* tylko adresy RPC i klucze API.
- Osobny klucz dla każdej roli: wdrożenie, bot, sterowanie ceną; minimalne salda.
- Wdrożenia wykonywane lokalnie, nie z CI. Jeśli kiedyś z CI, to tylko ręcznie uruchamianym workflow z chronionym środowiskiem i kluczem używanym wyłącznie na testnecie.
- Plik *.gitignore* wyklucza katalogi z kluczami i lokalnymi danymi; skaner sekretów w CI.

15.6 Oznaczenia i granice wdrożenia testowego

- Token ma w nazwie i symbolu sufiks TEST, np. „Redoubt USD TEST” / rUSD-TEST.
- README zaczyna się od informacji, że to demonstracja techniczna na testnecie, bez wartości i bez oferty dla kogokolwiek.
- Brak publicznego interfejsu do emisji; demonstracja odbywa się skryptami i panelem tylko do odczytu.
- Wdrożenie testowe dowodzi weryfikowalności i działania operacyjnego. Nie dowodzi ekonomii — tej dowodzą eksperymenty na forku.

16 Strategia testów

16.1 Warstwy

Warstwa	Narzędzia	Co sprawdza	Kiedy
Jednostkowe	Foundry	Każda funkcja, przejście stanu, revert	Każdy PR
Stanowe (niezmienniki)	Foundry invariant z handlerami	I1–I14 przy losowych sekwencjach	PR krótko, nocą długo
Fuzzing długi	Medusa	Te same niezmienniki, miliony wywołań	Nocą
Symboliczne	Halmos	Zaokrąglenia, zachowanie wartości w operacjach	PR
Różnicowe	Hypothesis + web3.py + anvil	Zgodność modelu Pythona z kontraktami	PR krótko, nocą długo
Na forku	Foundry fork, skrypty	Prawdziwe feedy, WETH, pule	Ręcznie i przed publikacją
Scenariuszowe	Skrypty S1–S9	Zachowanie systemu w krachach	Przed raportem
Statyczne	Slither, Aderyn	Znane klasy błędów, z plikiem triażu	Każdy PR
Chaos	Wyłączanie RPC, opóźnienia, reorganizacje	Bot i monitor	Przed wdrożeniem
Dymne	SmokeTest.s.sol	Świeże wdrożenie na testnecie	Po każdym wdrożeniu

16.2 Niezmienniki

ID	Niezmiennik
I1	Saldo WETH rdzenia = suma zabezpieczeń pozycji + zabezpieczenie w aukcjach + nadwyżki i rezerwy do odbioru.
I2	Podaż stablecoina = suma długów pozycji + dług w aukcji + zobowiązania puli − deficyt pokryty rezerwą (w ujęciu księgowym specyfikacji).
I3	Suma wartości depozytów w puli ≤ saldo puli; żadna wypłata nie przekracza udziału.
I4	Likwidacja nie obniża TCR, chyba że pozycja była bez pokrycia — wtedy różnica jest w deficycie.
I5	Wykup nie tworzy pozycji z ICR poniżej MCR ani z długiem poniżej minimum (poza zerem).
I6	Zabezpieczenie systemu + zapisany deficyt pokrywa zobowiązania wobec posiadaczy.
I7	Nadwyżka po likwidacji nigdy nie trafia do kogoś innego niż właściciel pozycji.
I8	Po dowolnej sekwencji operacji brak niedoboru z tytułu zaokrągleń.
I9	Spłata i dopłata są możliwe w każdym stanie poza zakończonym podziałem awaryjnym.
I10	W trybie awaryjnym nie powstaje nowy dług, a suma roszczeń ≤ dostępne zabezpieczenie.
I11	Parametry systemu nie zmieniają się nigdy po wdrożeniu.
I12	Operacje wymagające ceny nie wykonują się przy nieużytecznym odczycie.
I13	Lista pozycji jest posortowana według NICR, z uwzględnieniem nagród oczekujących.
I14	Stan odtworzony ze zdarzeń jest równy stanowi odczytanemu z kontraktów.

Dokładne wzory niezmienników I1 i I2 wynikają z księgowania opisanego w specyfikacji; ich sformułowanie jest pierwszym zadaniem etapu 0 i podlega przeglądowi przed implementacją.

16.3 Testy stanowe w Foundry

Wzorzec handlera (skrót)

```
contract Handler is Test {
    VaultManager vm_; StabilityPool pool; MockAggregator oracle;
    address[] actors; uint256 public ghostMinted; uint256 public ghostBurned;

    function open(uint256 a, uint256 coll, uint256 debt) external {
        address who = actors[bound(a, 0, actors.length - 1)];
        coll = bound(coll, 1e15, 1_000e18); debt = bound(debt, MIN_DEBT, 2_000_000e18);
        vm.prank(who);
        try vm_.openPosition(coll, debt) { ghostMinted += debt; } catch {}
    }
    function movePrice(uint256 bps) external {             // ruch ceny w granicach ±30%
        oracle.set(oracle.price() * bound(bps, 7_000, 13_000) / 10_000);
    }
    function warp(uint256 s) external { vm.warp(block.timestamp + bound(s, 1, 7 days)); }
    // liquidate, redeem, deposit, withdraw, repay ...
}

contract Invariants is Test {
    function setUp() public { /* wdrożenie, aktorzy */ targetContract(address(handler)); }
    function invariant_I1_collateralConservation() public view { /* ... */ }
    function invariant_I8_noRoundingDeficit() public view { /* ... */ }
}
```

Konfiguracja: *[invariant] runs = 256*, *depth = 200*, *fail_on_revert = false* przy PR; w nocnym przebiegu wartości wielokrotnie większe. Zmienne „duchy” (ghost) w handlerze śledzą to, czego kontrakt nie przechowuje wprost, np. łączną emisję, co pozwala sprawdzać niezmienniki globalne niezależnie od implementacji.

16.4 Dowody symboliczne

- Funkcje zaokrągleń: wynik w górę jest ≥ wyniku dokładnego, wynik w dół ≤, różnica < 1 jednostka.
- Pojedyncze umorzenie w puli: suma wartości depozytów po operacji = przed − umorzony dług.
- Podział pro rata: suma wypłat dla dowolnego podziału podaży ≤ pula zabezpieczenia.
- Naliczenie odsetek: podaż po operacji = podaż przed + przyrost długu.

Arytmetyki produkt–suma w pełnej sekwencji nie dowodzimy symbolicznie — pokrywa ją fuzzing i test różnicowy.

16.5 Test różnicowy

Harness w Pythonie korzysta z testów stanowych biblioteki Hypothesis: maszyna reguł generuje sekwencje operacji (otwarcie, pożyczka, ruch ceny, upływ czasu, likwidacja, wykup, wpłata do puli), wykonuje każdą na modelu i na kontraktach w anvilu przez web3.py, a po każdym kroku porównuje migawki stanu. Przy niezgodności Hypothesis automatycznie **skraca** sekwencję do najkrótszej, która nadal pokazuje błąd — to zamienia godziny debugowania w minuty.

Szkic maszyny stanów (Python)

```
class Differential(RuleBasedStateMachine):
    @rule(coll=collaterals, debt=debts)
    def open(self, coll, debt):
        r_model = self.model.open(coll, debt)
        r_chain = self.chain.open(coll, debt)          # web3.py -> anvil
        assert r_model.ok == r_chain.ok

    @rule(bps=st.integers(7000, 13000))
    def move_price(self, bps): self.model.set_price(bps); self.chain.set_price(bps)

    @invariant()
    def states_match(self):
        assert close(self.model.snapshot(), self.chain.snapshot(), tol=ROUNDING_TOL)
```

16.6 Ciągła integracja

Wyzwalacz	Zadania
Każdy PR	forge fmt --check, build, testy jednostkowe, krótkie niezmienniki, Halmos, Slither, Aderyn, pytest, krótki test różnicowy, skaner sekretów
Nocą	Długie kampanie Medusy i niezmienników, długi test różnicowy, przebudowa wykresów raportu na danych z repozytorium
Ręcznie	Scenariusze na forku, wdrożenie testowe (lokalnie), test dymny

17 Bezpieczeństwo i model zagrożeń

17.1 Założenia zaufania

Element	Czemu ufamy	Czemu nie ufamy
Pożyczkobiorca	Niczemu poza podpisem	Może nie reagować albo szukać zysku na krawędziach
Likwidator	Że działa dla zysku	Że przyjdzie, gdy jest nieopłacalnie
Depozytariusz	Że maksymalizuje zysk	Że zostanie w puli w złym momencie
Wyrocznia	Że zwykle podaje sensowną cenę	Że zawsze jest świeża i poprawna
Sekwencer Base	Że zwykle działa	Dostępności i neutralności kolejności
Autor kontraktów	Niczemu — brak ról administracyjnych	Że nie popełni błędu

17.2 Rejestr zagrożeń

Zagrożenie	Mitygacja	Weryfikacja
Manipulacja lub awaria wyroczni	Świeżość, granice, try/catch, blokada operacji, tryb awaryjny	Testy gałęzi adaptera, S6
Likwidacja na starych danych po przerwie sekwencera	Karencja	S7
Pozycje nieopłacalne do likwidacji	Minimalny dług i rezerwa gazowa	I5, testy jednostkowe
Arbitraż starej ceny w trybie awaryjnym	Podział pro rata w naturze	I10
Run przy niedoborze	Brak kolejności w podziale; okno spłat	Scenariusze skrajne
Atak na pierwszy depozyt w puli	Wirtualne udziały, minimalny depozyt	Testy, Halmos
Błędy precyzji produkt–suma	Epoki, skale, korekta błędu	I3, Medusa, test różnicowy
Zapchanie listy pozycji	Limit iteracji podawany przez wywołującego	Testy gazowe
Wyprzedzenie zakupu w aukcji	Maksymalna cena w buy	Testy jednostkowe
Samolikwidacja z zyskiem	Nadwyżka wraca do właściciela; premia ograniczona	I7
Atak wolumenowy na wykup	Bazowa stopa rosnąca z wolumenem	Testy, scenariusz
Kumulacja zaokrągleń	Kierunki zaokrągleń ze specyfikacji	I8, Halmos
Reentrancy	Checks-effects-interactions, blokada, tylko WETH	Slither, testy
Wyciek klucza testowego	Magazyn kluczy, osobne role, minimalne salda	Skaner sekretów

17.3 Granice

Testnetowy protokół bez audytu zewnętrznego nie jest gotowy do obsługi cudzych środków. Emisja stablecoina skierowana do użytkowników podlega regulacjom — w Unii Europejskiej przepisom MiCA, w Stanach Zjednoczonych przepisom stanowym i federalnym. Dokument nie jest poradą prawną; wniosek praktyczny brzmi: projekt pozostaje demonstracją techniczną, bez oferowania tokena komukolwiek.

18 Stos technologiczny i infrastruktura

18.1 Kontrakty

Element	Wybór	Rola
Język	Solidity 0.8.x (wersja przypięta w foundry.toml)	Kontrakty rdzenia
Framework	Foundry: forge, anvil, cast, chisel	Kompilacja, testy, fork, skrypty, wdrożenia
Biblioteki	OpenZeppelin Contracts 5.x (ERC20, SafeERC20, ReentrancyGuard); solady albo prb-math (mulDiv z kierunkiem zaokrąglenia)	Sprawdzone prymitywy
Interfejsy zewnętrzne	AggregatorV3Interface (Chainlink), IWETH, kontrakty kwotujące DEX	Wyrocznia, zabezpieczenie, pomiar płynności
Analiza	Slither, Aderyn	Statyczna
Fuzzing	Foundry invariant, Medusa (opcjonalnie Echidna)	Stanowy
Symbolicznie	Halmos (opcjonalnie Certora)	Dowody prostych własności
Gaz	forge snapshot	Regresje kosztu operacji

18.2 Model, dane i symulacja

Element	Wybór	Rola
Środowisko	Python 3.12, uv, pyproject.toml	Powtarzalne zależności
Obliczenia	numpy, pandas (albo polars), scipy	Model, statystyki
Bootstrap	pakiet arch (bootstrap stacjonarny i blokowy)	Generator ścieżek
Dane	pyarrow (Parquet), httpx, Dune API lub RPC archiwalne	Pobieranie i przechowywanie
Łańcuch	web3.py, eth-abi	Sterowanie anvilem, forkami i botem
Testy	pytest, Hypothesis (w tym testy stanowe)	Model i test różnicowy
Wykresy	matplotlib	Raport
Notatniki	Jupyter tylko do eksploracji	Wyniki raportu wyłącznie ze skryptów

18.3 Operacje i publikacja

Element	Wybór	Rola
RPC	Dostawca z dostępem archiwalnym dla Base; publiczny RPC dla Base Sepolia	Forki i wdrożenia
Eksplorator	Basescan (klucz API)	Weryfikacja kodu
Bot	Python, web3.py, logowanie strukturalne, alerty	Likwidacje na forku i testnecie
Indeksator	Python, PostgreSQL	Stan ze zdarzeń, monitor niezmienników
Panel (opcja)	Rails 8, Hotwire	Podgląd tylko do odczytu
Automatyzacja	just albo Makefile, GitHub Actions	Jedno polecenie na każdy krok
Kontenery (opcja)	Docker Compose	Anvil, PostgreSQL, bot, indeksator razem

18.4 Repozytorium

Struktura monorepo

```
redoubt/
├── contracts/                        # Foundry
│   ├── src/
│   │   ├── core/      VaultManager.sol  SortedPositions.sol  StableToken.sol  OracleAdapter.sol
│   │   ├── liquidation/  StabilityPool.sol  DutchAuctionHouse.sol  ILiquidationStrategy.sol
│   │   ├── redemption/   RedemptionModule.sol
│   │   ├── safety/       DeficitLedger.sol  ShutdownModule.sol
│   │   ├── lib/          WadRayMath.sol  Rounding.sol
│   │   └── mocks/        MockAggregator.sol  MockSequencerFeed.sol  ReplayAggregator.sol
│   ├── test/  unit/  invariant/  symbolic/  fork/  scenarios/
│   ├── script/  Deploy.s.sol  DeployFork.s.sol  SmokeTest.s.sol  Scenario.s.sol
│   └── foundry.toml  medusa.json
├── model/          protocol/  oracle.py  liquidity.py  costs.py
├── agents/         borrowers.py  liquidators.py  depositors.py
├── paths/          historical.py  bootstrap.py
├── experiments/    sweep.py  calibrate.py  liquidity.py  configs/S1.yaml ... S9.yaml
├── differential/   machine.py  chain.py
├── data/           cards/  raw/  processed/  fetch_*.py
├── bot/  indexer/  dashboard/
├── deployments/    31337.json  84532.json
├── reports/        figures/  report.md
├── docs/           spec.md  invariants.md  preregistration.md  adr/  threat-model.md
├── CLAUDE.md  justfile  pyproject.toml  README.md
```

contracts/foundry.toml (szkic)

```
[profile.default]
src = "src"; test = "test"; script = "script"
solc_version = "0.8.x"            # przypięta konkretna wersja
optimizer = true; optimizer_runs = 200
fs_permissions = [{ access = "read-write", path = "../deployments" }]

[invariant]
runs = 256; depth = 200; fail_on_revert = false

[profile.nightly.invariant]
runs = 4096; depth = 500

[rpc_endpoints]
base = "${BASE_ARCHIVE_RPC}"
base_sepolia = "${BASE_SEPOLIA_RPC}"

[etherscan]
base_sepolia = { key = "${BASESCAN_API_KEY}", chain = 84532 }
```

justfile (najważniejsze polecenia)

```
check:        forge fmt --check && forge build && forge test && uv run pytest -q
invariants:   FOUNDRY_PROFILE=nightly forge test --match-path "test/invariant/*"
medusa:       cd contracts && medusa fuzz
diff:         uv run pytest differential -q
data:         uv run python -m data.fetch_all && uv run python -m data.validate
calibrate:    uv run python -m experiments.calibrate
sweep id:     uv run python -m experiments.sweep --config experiments/configs/{{id}}.yaml
report:       uv run python -m experiments.report
fork block:   anvil --fork-url $BASE_ARCHIVE_RPC --fork-block-number {{block}} --chain-id 8453
deploy-sepolia: cd contracts && forge script script/Deploy.s.sol:Deploy --rpc-url base_sepolia \
                --account redoubt-deployer --broadcast --verify
smoke:        cd contracts && forge script script/SmokeTest.s.sol --rpc-url base_sepolia \
                --account redoubt-deployer --broadcast
```

19 Plan realizacji krok po kroku

Plan jest zadaniowy i zależnościowy, bez kalendarza. Każdy krok ma artefakt i warunek ukończenia. **Minimalna wersja publikowalna kończy się na etapie 5** — jeśli projekt zatrzyma się tam, nadal ma wynik do pokazania. Etapy 6–9 są rozszerzeniem.

Etap	Rezultat	Zależy od
0. Fundamenty	Repozytorium, CI, specyfikacja, niezmienniki, ADR, szkic prerejestracji	—
1. Dane	Zwalidowane zestawy z kartami	0
2. Rdzeń	Pozycje, odsetki, lista, wykup, wyrocznia	0
3. Pula i redystrybucja	Wdrożenie A kompletne	2
4. Model i test różnicowy	Model zgodny z kontraktami; modele otoczenia	1, 3
5. Kalibracja i wyniki (MVP)	Raport cząstkowy dla wdrożenia A	4
6. Aukcja	Wdrożenie B i porównanie	5
7. Tryb awaryjny	Domknięty wodospad strat	3
8. Hartowanie i wdrożenia	Fork, bot, monitor, Base Sepolia	6, 7
9. Publikacja	Raport końcowy, README, wideo, wpis	8

19.1 Etap 0: fundamenty

Cel. ustalić, co ma być zbudowane i udowodnione, zanim powstanie kod trudny do zmiany.

Krok	Co zrobić	Artefakt	Gotowe, gdy
0.1	Monorepo: forge init, uv init, katalogi, justfile, .gitignore, licencja	Szkielet	just check przechodzi na pustym projekcie
0.2	CI: workflow dla PR i nocny; cache Foundry i uv; skaner sekretów	.github/workflows	Zielony przebieg
0.3	Specyfikacja: stan, wzory WAD/RAY, kierunki zaokrągleń, przejścia, algorytmy z rozdz. 8	docs/spec.md	Da się z niej pisać kod bez zgadywania
0.4	Niezmienniki I1–I14 z dokładnymi wzorami	docs/invariants.md	Każdy ma formułę i docelowy test
0.5	Wpisy ADR 001–015	docs/adr/	Każda decyzja z rozdz. 6 ma wpis
0.6	Szkic prerejestracji (zatwierdzany w kroku 5.4)	docs/preregistration.md	Hipotezy i metryki spisane
0.7	Plik zasad dla Claude Code (rozdz. 20)	CLAUDE.md	Zasady obowiązują od pierwszej sesji

> **Kryterium ukończenia etapu**
> Specyfikacja i niezmienniki przeszły przegląd, a repozytorium ma działające CI.

19.2 Etap 1: dane

Cel. zebrać i zwalidować dane, zanim zaczną od nich zależeć wyniki.

Krok	Co zrobić	Artefakt	Gotowe, gdy
1.1	Pobranie minutowych cen ETH/USD z dwóch giełd za kilka lat	data/raw, karta	Skrypt odtwarza plik od zera
1.2	Walidacja: luki, duplikaty, strefy czasowe, rozjazdy giełd	Raport walidacji	Każda anomalia opisana
1.3	Historia feedu ETH/USD na Base: aktualizacje, odstępy, ruchy	Parquet, karta	Znany rozkład odstępów i progów
1.4	Historia feedu dostępności sekwencera	Parquet, karta	Lista okresów przerw
1.5	Zdarzenia likwidacji z jednego–dwóch protokołów na Base	Parquet, karta	Pełna lista w wybranym okresie
1.6	Rekonstrukcja stanu pozycji w oknach gwałtownych ruchów	Tabela przekroczeń progu	Znane też pozycje niezlikwidowane
1.7	Wybór bloków spokojnych i stresowych do pomiaru płynności	Lista bloków	Co najmniej po kilka z każdego reżimu
1.8	Szeregi kosztów transakcji L2 i L1	Parquet, karta	Oba reżimy pokryte

> **Kryterium ukończenia etapu**
>
> Każdy zestaw ma kartę, sumę kontrolną i skrypt pobierający; dane surowe nie są edytowane ręcznie.

19.3 Etap 2: rdzeń kontraktów

Cel. działający protokół bez likwidacji: pozycje, odsetki, lista, wykup, wyrocznia.

Krok	Co zrobić	Artefakt	Gotowe, gdy
2.1	Biblioteka WAD/RAY: mulDiv w górę i w dół, rpow	lib/WadRayMath.sol	Testy + dowody Halmos zaokrągleń
2.2	StableToken z uprawnieniami mennicy	core/StableToken.sol	Tylko rdzeń emituje i spala
2.3	Mocki: MockAggregator, MockSequencerFeed, ReplayAggregator	mocks/	Interfejs zgodny z Chainlink
2.4	OracleAdapter: świeżość, granice, sekwencer, karencja, try/catch	core/OracleAdapter.sol	Test dla każdej gałęzi
2.5	SortedPositions: wstawianie z podpowiedzią, usuwanie, przechodzenie	core/SortedPositions.sol	I13 przy losowych wstawieniach
2.6	VaultManager: otwarcie, dopłata, pożyczka, spłata, wypłata, zamknięcie, accrue, minimalny dług, rezerwa	core/VaultManager.sol	Testy przejść; I1, I2, I9, I11, I12
2.7	RedemptionModule: kolejka według NICR, pomijanie pozycji pod progiem, bazowa stopa	redemption/	I5 przy losowych wykupach
2.8	Handler i testy stanowe; pierwsza kampania Medusy	test/invariant/	Brak naruszeń
2.9	Skrypt wdrożenia lokalnego (CREATE2)	script/Deploy.s.sol	Stałe adresy na anvilu

Kryterium ukończenia etapu

Otwarcie, spłata i wykup przechodzą; niezmienniki rdzenia zielone w długiej kampanii fuzzingu.

19.4 Etap 3: pula i redystrybucja (wdrożenie A)

Cel. pierwszy kompletny mechanizm likwidacji.

Krok	Co zrobić	Artefakt	Gotowe, gdy
3.1	StabilityPool: produkt–suma, epoki, skale, korekta błędu, zysk z odsetek, wirtualne udziały	liquidation/StabilityPool.sol	I3 w osobnej kampanii
3.2	Likwidacja przez pulę z rekompensatą z rezerwy gazowej	VaultManager + pula	Pozycja pod progiem likwidowana w jednej transakcji
3.3	Redystrybucja: akumulatory L, nagrody oczekujące w ICR i NICR	VaultManager	I13 po redystrybucjach
3.4	DeficitLedger: deficyt, rezerwa z odsetek	safety/DeficitLedger.sol	I6
3.5	Rozszerzenie handlera, I4 i I7, kampanie nocne	test/invariant/	Brak naruszeń

Kryterium ukończenia etapu

Wdrożenie A przechodzi wszystkie niezmienniki przy sekwencjach z likwidacjami i redystrybucją.

19.5 Etap 4: model i test różnicowy

Cel. szybki model zgodny z kontraktami i modele otoczenia.

Krok	Co zrobić	Artefakt	Gotowe, gdy
4.1	Model protokołu w Pythonie: rdzeń, lista, wykup, pula, redystrybucja	model/protocol/	Testy jednostkowe modelu
4.2	Most do anvila i migawki stanu	differential/chain.py	Migawka obejmuje wszystkie pola niezmienników
4.3	Maszyna stanów Hypothesis i wpięcie do CI	differential/machine.py	Zgodność na ≥ 200 sekwencjach
4.4	Model wyroczni z danych 1.3	model/oracle.py	Odtwarza historię feedu na próbie testowej
4.5	Pomiary płynności na forkach z bloków 1.7; dopasowanie krzywej poślizgu	model/liquidity.py, wyniki pomiarów	Krzywe dla obu reżimów
4.6	Model kosztów transakcji	model/costs.py	Reżimy L2 i L1 z danych 1.8

Kryterium ukończenia etapu

Test różnicowy jest zielony w CI, a modele otoczenia są dopasowane do danych z etapu 1.

19.6 Etap 5: kalibracja i pierwsze wyniki (MVP)

Cel. wyprodukować pierwszy publikowalny wynik dla wdrożenia A.

Krok	Co zrobić	Artefakt	Gotowe, gdy
5.1	Agenci pożyczkobiorców i depozytariuszy z parametrem ucieczki	agents/	Testy zachowań
5.2	Agent likwidatora i kalibracja na danych 1.5–1.6	experiments/calibrate.py	Rozkład opóźnień w przedziale ufności danych
5.3	Generator ścieżek: bootstrap blokowy i ścieżki historyczne	paths/	Zachowane statystyki zwrotów i skupianie zmienności
5.4	Zatwierdzenie prerejestracji commitem przed przemiataniem	docs/preregistration.md	Commit w historii
5.5	Przemiatanie wdrożenia A: S1, S2, S4, S6, S7, S9	Wyniki z ziarnami	Komplet zgodnie z prerejestracją
5.6	Weryfikacja wybranych punktów na forku odtwarzanym i historycznym (S3)	Porównanie model kontra fork	Różnice opisane
5.7	Raport cząstkowy, README, wpis techniczny	reports/, README.md	Wykresy odtwarzalne jednym poleceniem

Kryterium ukończenia etapu

Opublikowany raport cząstkowy z buforem wyprowadzonym dla apetytu na ryzyko i mapą obszaru bezpiecznego.

19.7 Etap 6: aukcja (wdrożenie B)

Cel. druga strategia i porównanie.

Krok	Co zrobić	Artefakt	Gotowe, gdy
6.1	DutchAuctionHouse: krzywa, buy z ceną maksymalną, reset, zakończenie, dług w aukcji	liquidation/DutchAuctionHouse.sol	Testy jednostkowe
6.2	Niezmienniki dla wdrożenia B, kampanie nocne	test/invariant/	Brak naruszeń
6.3	Model aukcji i licytanta z pożyczką błyskawiczną; rozszerzenie testu różnicowego	model/, differential/	Zgodność dla B
6.4	Przemiatanie B, porównanie z A, weryfikacja hipotez	Wyniki, tabela porównawcza	Hipotezy rozstrzygnięte według reguł

Kryterium ukończenia etapu

Komplet wyników dla obu wdrożeń w identycznych warunkach.

19.8 Etap 7: tryb awaryjny

Cel. domknąć zachowanie w sytuacjach skrajnych.

Krok	Co zrobić	Artefakt	Gotowe, gdy
7.1	ShutdownModule: warunki, okno spłat, podział pro rata	safety/ShutdownModule.sol	Testy każdej przyczyny wyzwolenia
7.2	I10, scenariusze skrajne, model w Pythonie	Testy, model	Uporządkowane zamknięcie w każdym scenariuszu

> **Kryterium ukończenia etapu**
> Każda strata w każdym scenariuszu trafia na właściwy szczebel wodospadu.

19.9 Etap 8: hartowanie i wdrożenia

Cel. przygotować system do publicznego pokazania.

Krok	Co zrobić	Artefakt	Gotowe, gdy
8.1	Długie kampanie, triaż Slithera i Aderyna	Plik triażu	Każde ostrzeżenie wyjaśnione
8.2	Przegląd według listy kontrolnej z publicznych znalezisk audytowych	docs/threat-model.md	Lista przejrzana punkt po punkcie
8.3	Skrypty forka i scenariusz S3 end-to-end z botem	script/DeployFork.s.sol	Bot likwiduje bez udziału człowieka
8.4	Bot likwidacyjny: odpytywanie, opłacalność, ponowienia, alerty	bot/	Testy chaosu
8.5	Indeksator i monitor z I14	indexer/	Stan ze zdarzeń równy odczytom
8.6	Base Sepolia według procedury 15.4: wdrożenie, weryfikacja, manifest, test dymny	deployments/84532.json	Kontrakty zweryfikowane, test dymny zielony

> **Kryterium ukończenia etapu**
> Publiczne wdrożenie działa z botem i monitorem; fork pokazuje system w realnym otoczeniu.

19.10 Etap 9: publikacja

Cel. zamienić wyniki w materiał oceniany bez rozmowy z autorem.

Krok	Co zrobić	Artefakt	Gotowe, gdy
9.1	Raport końcowy według szkieletu z rozdz. 14.4	reports/report.md	Każda liczba ma źródło w danych i konfiguracji
9.2	README: architektura, uruchomienie, adresy, ograniczenia	README.md	Osoba z zewnątrz uruchamia demo
9.3	Wideo: 1 minuta przebiegu krachu, 5 minut metody i wyników	Nagrania	Nic nie jest udawane
9.4	Wpis techniczny o najciekawszym znalezisku	Artykuł	Opublikowany z linkiem do repozytorium

Krok	Co zrobić	Artefakt	Gotowe, gdy
9.5	Opcje: panel, drugie aktywo, analiza kotwicy, test wobec Liquity	Lista	Decyzja, co dalej

Kryterium ukończenia etapu

Projekt jest publiczny, odtwarzalny i opisany razem z ograniczeniami.

20 Praca z Claude Code

Projekt finansowy prowadzony z agentem ma jedno szczególne ryzyko: agent piszący implementację i testy jednocześnie potrafi dopasować jedno do drugiego, a błąd w specyfikacji „naprawić" zmianą testu. Zasady poniżej są po to, żeby poprawność nie ginęła niezauważenie.

20.1 Zasady

- **Źródło prawdy to specyfikacja i niezmienniki** w repozytorium, nie rozmowa. Każda sesja zaczyna się od ich wczytania.
- **Jeden krok planu na sesję**, z jasnym artefaktem i warunkiem ukończenia z rozdziału 19.
- **Najpierw testy, potem kod**, a testy wynikają ze specyfikacji, nie z implementacji.
- **Niezmiennika nie zmienia się po to, żeby test przeszedł.** Złamany niezmiennik to błąd w kodzie albo w specyfikacji; zmiana specyfikacji wymaga nowego wpisu ADR.
- **Liczby w raporcie powstają wyłącznie ze skryptów**, nigdy z ręcznie wklejonych wyników.
- **Przegląd każdej zmiany przed zatwierdzeniem**, ze szczególną uwagą na kierunki zaokrągleń, uprawnienia i pętle.

CLAUDE.md (szkic)

```
# Redoubt — zasady pracy

## Źródła prawdy
- docs/spec.md — specyfikacja; docs/invariants.md — niezmienniki I1–I14
- docs/adr/ — decyzje; zmiana decyzji = nowy wpis, nigdy edycja starego
- docs/preregistration.md — po zatwierdzeniu tylko datowane dopiski

## Reguły
- Jeden krok planu (rozdz. 19 blueprintu) na zadanie; zacznij od testów.
- Nie zmieniaj niezmiennika ani testu, żeby przeszedł. Zgłoś konflikt i zatrzymaj się.
- Każde dzielenie ma jawny kierunek zaokrąglenia zgodny z tabelą w spec.md.
- Żadnych ról administracyjnych, funkcji ustawiających parametry ani proxy.
- Zmiana w kontraktach wymaga odpowiadającej zmiany w modelu Pythona (test różnicowy).
- Wyniki i wykresy tylko ze skryptów w experiments/; nie edytuj plików w reports/ ręcznie.
- Klucze prywatne nigdy w plikach ani w poleceniach; tylko --account z magazynu Foundry.

## Polecenia
- just check · just invariants · just diff · just sweep S2 · just report
```

21 Mocne i słabe strony

21.1 Mocne strony

- **Rzadka kombinacja**: kontrakty i ilościowa analiza ryzyka.
- **Wyniki nie do skopiowania**: mapy parametrów z kalibrowanej symulacji i pomiarów na forku.
- **Rdzeń DeFi**: likwidacje i wyrocznie to temat rozmów w każdym protokole pożyczkowym.
- **Rzetelność widoczna w repozytorium**: prerejestracja, karty danych, test różnicowy.
- **Realne otoczenie** dzięki forkom zamiast fikcyjnej płynności testnetu.
- **Trwałość**: mechanika CDP nie zestarzeje się razem z modą.

21.2 Słabe strony

- **Wydeptana ścieżka**: różnica musi siedzieć w raporcie, więc raport musi być dobry.
- **Rekonstrukcja stanu do kalibracji** jest pracochłonna i może wymagać uproszczeń.
- **Dostęp archiwalny RPC** bywa płatny lub limitowany.
- **Wyniki warunkowe**: model agentów pozostaje hipotezą.
- **Duży zakres** — stąd granica MVP po etapie 5.

21.3 Ryzyka

Ryzyko	Prawdop.	Wpływ	Mitygacja
Rozlanie zakresu	wysokie	wysoki	Granica MVP po etapie 5
Rozjazd modelu i kontraktów	średnie	wysoki	Test różnicowy w CI od etapu 4
Trudna rekonstrukcja danych do kalibracji	wysokie	średni	Okna wokół ruchów; gotowe tabele zdarzeń; jawne uproszczenia
Limity RPC archiwalnego	średnie	średni	Buforowanie forków; ograniczona liczba bloków
Błąd precyzji w puli	średnie	wysoki	Osobna kampania, test różnicowy, wzorzec referencyjny
Wyniki banalne	średnie	średni	Liczby i granice mają wartość także przy potwierdzeniu intuicji
Zmęczenie projektem	średnie	wysoki	Pierwsze wyniki już w etapie 5

22 Demonstracja i publikacja

22.1 Przebieg demonstracyjny (1 minuta)

Czas	Co widać	Co to pokazuje
0:00–0:10	Fork Base z blokiem z sierpnia 2024; kilka pozycji	Realne otoczenie
0:10–0:25	Odtwarzana ścieżka krachu; pozycje schodzą pod próg	Reakcja na prawdziwe ruchy
0:25–0:40	Bot likwiduje przez pulę; zdarzenia i monitor niezmienników	Domknięcie bez człowieka
0:40–0:55	Wdrożenie B: aukcja, reset, poślizg na prawdziwej puli DEX	Różnica mechanizmów
0:55–1:00	Mapa obszaru bezpiecznego z raportu	Parametry wyprowadzone, nie przepisane

22.2 Wersja rozszerzona i publikacja

Pięć minut: metoda, kalibracja, porównanie mechanizmów, uruchomienie niezmienników i minuta o ograniczeniach. Publikacja: repozytorium z README i wideo na górze, zweryfikowane kontrakty na Base Sepolia, wpis techniczny o jednym znalezisku na blogu autora oraz udostępnienie w społecznościach DeFi i ryzyka.

23 Otwarte pytania

1. **Reguła socjalizacji**: redystrybucja czy deficyt jako domyślna?
2. **Tryb awaryjny przy działającej wyroczni**: zostać przy podziale pro rata czy rozliczać pozycje po cenie z chwili wyzwolenia, zwracając nadwyżkę pożyczkobiorcom?
3. **Ucieczka z puli**: opóźnienie wypłat, premia za lojalność czy brak ingerencji?
4. **Ułamek domknięcia**: pełna czy częściowa likwidacja jako domyślna?
5. **Kształt krzywej aukcji**: wykładnicza czy schodkowa, z jakim czasem maksymalnym?
6. **Okres karencji sekwencera**: jak długi?
7. **Apetyt na ryzyko**: jaka wartość domyślna w prerejestracji?
8. **Zakres kalibracji**: ile protokołów i jaki okres, by wynik nie był artefaktem jednego rynku?
9. **Drugie aktywo** po MVP czy dopracowanie raportu?
10. **Nazwa**: zostaje Redoubt?

24 Słownik pojęć

Pojęcie	Znaczenie
CDP	Pozycja zabezpieczona długiem: zablokowane zabezpieczenie i wyemitowany stablecoin
ICR, TCR, MCR	Wskaźnik pozycji, systemu i minimalny dopuszczalny
NICR	Zabezpieczenie / dług znormalizowany; kolejność niezależna od ceny i odsetek
WAD, RAY	Skale stałoprzecinkowe 10^18 i 10^27
Dług znormalizowany	Dług podzielony przez indeks odsetek w chwili zaciągnięcia
Produkt–suma	Algorytm puli stabilności w czasie stałym: akumulatory P i S
Redystrybucja	Rozłożenie niepokrytego długu na pozostałe pozycje przez akumulatory L
Dług w aukcji	Dług zlikwidowanej pozycji oczekujący na rozliczenie aukcji
Rezerwa gazowa	WETH wpłacone przy otwarciu na rekompensatę dla likwidatora
Podpowiedź	Sąsiedzi na liście wyliczeni poza łańcuchem, przyspieszający wstawianie
Karencja	Czas po powrocie sekwencera bez likwidacji
Fork historyczny	Kopia stanu Base mainnet z wybranego bloku na lokalnym węźle
ReplayAggregator	Kontrakt-wyrocznia odtwarzający historyczną ścieżkę ceny
Blokowy bootstrap	Losowanie bloków historycznych zwrotów, zachowujące skupianie zmienności
Prerejestracja	Zapisanie hipotez i reguł oceny przed eksperymentem
Błąd przeżywalności	Zniekształcenie wynikające z obserwowania tylko zdarzeń, które zaszły
Test różnicowy	Porównanie dwóch implementacji tych samych reguł na tych samych danych
Manifest wdrożenia	Plik z adresami, blokiem, commitem i parametrami wdrożenia

25 Źródła i materiały

Lista kierunkowa; dokładne adresy, wersje i daty dostępu trafiają do repozytorium. Adresy feedów i pul na Base należy pobierać z oficjalnej dokumentacji tuż przed użyciem.

25.1 Protokoły referencyjne

- Liquity v1: whitepaper, kod i praca „Scalable Reward Distribution with Compounding Stakes” — pula stabilności, redystrybucja, lista posortowana z podpowiedziami, opłata wykupu.
- Liquity v2 (BOLD): monorepo i dokumentacja — nowoczesny CDP, wykupy, wyrocznie, wyłączanie gałęzi.
- MakerDAO / Sky: moduł aukcji Clipper i funkcje kształtu krzywej; potęgowanie w skali RAY; analizy marca 2020.
- Aave v3: wskaźnik zdrowia, ułamek domknięcia, bonus likwidacyjny, obsługa sekwencera na L2.

25.2 Infrastruktura i narzędzia

- Dokumentacja Chainlink Data Feeds i L2 Sequencer Uptime Feeds.
- Dokumentacja Base: sieci, predeploye (WETH), RPC, eksplorator.
- Foundry Book: testy stanowe, forki, skrypty, magazyn kluczy, weryfikacja kontraktów.
- Medusa, Halmos, Slither, Aderyn — dokumentacja narzędzi.
- Hypothesis: testy stanowe; pakiet arch: bootstrap blokowy i stacjonarny.

25.3 Bezpieczeństwo, ryzyko i dane

- Publiczne raporty audytów protokołów CDP i zgłoszenia z konkursów audytowych (kategorie: likwidacje, wyrocznie, precyzja).
- Publiczne analizy firm zajmujących się parametryzacją ryzyka w protokołach pożyczkowych.
- Prace badawcze o likwidacjach w DeFi i atakach na wyrocznie.
- Publiczne zbiory danych o zdarzeniach protokołów na Base.

A Dodatek: wzory i parametry

Symbol	Znaczenie	Wzór lub wartość startowa
ICR	Wskaźnik pozycji	zabezpieczenie · cena / (dług znorm. · indeks)
NICR	Wskaźnik nominalny	zabezpieczenie / dług znormalizowany
indeks	Indeks odsetek (RAY)	indeks · (1 + r)^Δt
MCR	Minimalny wskaźnik	premia + bufor luki + bufor poślizgu + margines
bufor luki	Ruch ceny w oknie t0–t3	kwantyl z bootstrapu dla apetytu na ryzyko; dolne ograniczenie 3·σ·√(T/rok)
V_{min}	Minimalna opłacalna pozycja	K / (p − s)
MIN_DEBT	Minimalny dług	z warunku opłacalności w reżimie L1 w krachu, z zapasem
P, S	Akumulatory puli	P ← P·(1 − D/T); S ← S + C·P/T
L_{coll}, L_{debt}	Akumulatory redystrybucji	L += wartość / Σudziałów
cena aukcji	Krzywa malejąca	cena startowa · e^(−k·t)
opłata wykupu	Minimum + bazowa stopa	bazowa stopa rośnie z wolumenem, zanika w czasie
podział awaryjny	Odbiór posiadacza	kwota · pula zabezpieczenia / niespłacona podaż

B Dodatek: szkic interfejsów

Interfejsy rdzenia (szkic)

```
interface IVaultManager {
    function openPosition(uint256 coll, uint256 debt, uint256 upperHint, uint256 lowerHint)
        external payable;                                  // payable: rezerwa gazowa
    function addCollateral(uint256 id, uint256 amount, uint256 upperHint, uint256 lowerHint) external;
    function repay(uint256 id, uint256 amount, uint256 upperHint, uint256 lowerHint) external;
    function borrow(uint256 id, uint256 amount, uint256 upperHint, uint256 lowerHint) external;
    function withdrawCollateral(uint256 id, uint256 amount, uint256 up, uint256 low) external;
    function close(uint256 id) external;
    function liquidate(uint256 id) external;               // każdy; wymaga użytecznej ceny
    function accrue() external;                            // każdy
    function icr(uint256 id) external view returns (uint256);
    function nicr(uint256 id) external view returns (uint256);
}

interface ILiquidationStrategy {                         // niezmienna w danym wdrożeniu
    struct Outcome { uint256 debtCleared; uint256 collUsed; uint256 deficit; }
    function liquidate(uint256 id, uint256 debt, uint256 coll) external returns (Outcome memory);
}

interface IStabilityPool {
    function deposit(uint256 amount) external;
    function withdraw(uint256 amount) external;
    function claimGains() external;                        // zabezpieczenie i odsetki
    function offset(uint256 debt, uint256 coll) external; // tylko VaultManager
}

interface IDutchAuctionHouse {
    function buy(uint256 auctionId, uint256 maxColl, uint256 maxPrice) external;
    function reset(uint256 auctionId) external;            // każdy, po czasie lub progu
    function price(uint256 auctionId) external view returns (uint256);
}

interface IRedemptionModule {
    function redeem(uint256 amount, uint256 maxIterations, uint256 maxFeeBps) external;
}

interface IShutdownModule {
    function check() external;                             // każdy; nieodwracalne wyzwolenie
    function claim(uint256 stableAmount) external;         // po oknie spłat, pro rata
}
```

C Dodatek: konfiguracja scenariusza

experiments/configs/S2.yaml

```
scenario: S2_krach_dobowy
seed: 20200312
environment: model                # model | fork_replay | fork_historical
price:
  source: historical              # historical | bootstrap
  dataset: eth_usd_1m_2020-03-10_2020-03-14
oracle:
  deviation_bps: from_data        # parametry feedu z historii na Base
  heartbeat_s: from_data
  sequencer: { downtime_windows: [], grace_period_s: 3600 }
costs: { regime: l2_cheap }       # l2_cheap | l1_crash
liquidity: { curve: fitted_crash_blocks, scale: 1.0 }
protocol:
  variant: A_stability_pool       # A_stability_pool | B_dutch_auction
  mcr_bps: 13500
  min_debt: 200
  gas_reserve_eth: 0.002
  rate_bps_year: 300
  pool_share_bps: 7500
  close_factor_bps: 10000
agents:
  borrowers:   { count: 500, icr_distribution: from_data,
                 reaction: calibrated }
  liquidators: { count: 12, latency: calibrated, min_profit_usd: 1.0 }
  depositors:  { pool_size_usd: 2000000, flight_sensitivity: 0.4 }
risk_appetite: { bad_debt_prob_per_position_year: 0.001 }
report: [bad_debt, bad_debt_frequency, time_underwater, value_leaked, pool_capital_cost,
         total_system_cost, unnecessary_liquidations]
```

D Dodatek: szkic skryptu wdrożenia

script/Deploy.s.sol (szkic)

```
contract Deploy is Script {
    function run() external {
        Config memory c = loadConfig(block.chainid);      // adresy feedów, WETH, parametry
        vm.startBroadcast();                               // konto z --account (magazyn kluczy)
        OracleAdapter oracle = new OracleAdapter{salt: SALT}(c.priceFeed, c.sequencerFeed, c.limits);

        // wdrożenie A — pula stabilności
        StableToken tokA = new StableToken{salt: SALT}("Redoubt USD TEST A", "rUSD-TEST-A");
        StabilityPool pool = new StabilityPool{salt: SALT}(tokA, c.weth);
        VaultManager vmA = new VaultManager{salt: SALT}(c.params, oracle, tokA, c.weth, pool);

        // wdrożenie B — aukcja holenderska
        StableToken tokB = new StableToken{salt: SALT_B}("Redoubt USD TEST B", "rUSD-TEST-B");
        DutchAuctionHouse dah = new DutchAuctionHouse{salt: SALT_B}(c.auction, oracle, tokB, c.weth);
        VaultManager vmB = new VaultManager{salt: SALT_B}(c.params, oracle, tokB, c.weth, dah);

        // powiązania jednorazowe w konstruktorach; brak funkcji ustawiających po wdrożeniu
        vm.stopBroadcast();
        writeManifest(block.chainid, oracle, vmA, pool, vmB, dah);   // deployments/<chainId>.json
    }
}
```

E Dodatek: szablon prerejestracji

docs/preregistration.md

```
# Prerejestracja — Redoubt
Data zatwierdzenia: RRRR-MM-DD   Commit: <hash>

## Pytania badawcze
Q1. Jaki MCR spełnia apetyt na ryzyko R dla wdrożenia A przy reżimach L2 i L1?
Q2. Który mechanizm ma niższy całkowity koszt systemu w scenariuszach S1–S9?

## Hipotezy
H1. Pula stabilności ma niższy dług nieściągalny niż aukcja w krachach krótszych niż X godzin.
H2. Aukcja ma niższy całkowity koszt systemu w osuwiskach dłuższych niż Y dni.

## Metryki (definicje z rozdz. 14.2 blueprintu)
## Apetyt na ryzyko: R = 0,1% pozycji z długiem nieściągalnym rocznie
## Scenariusze i siatka parametrów: S1–S9; MCR 110–200% co 5 pp; ...
## Reguły rozstrzygania
H1 potwierdzona, jeśli różnica median > 0 i 95% przedział z bootstrapu nie obejmuje zera.
## Zmiany po zatwierdzeniu
(tylko datowane dopiski z uzasadnieniem)
```